Unidad I — Calidad de Software

Apunte de estudio para el primer parcial

Aspectos Avanzados de Calidad de Software · ISI · UTN FRRe · 2026

Octubre de 2026

Índice

1. 0. Cómo usar este apunte **4**
- 1.1. 0.1 Qué cubre . . . 4
- 1.2. 0.2 Convenciones (cómo distinguir qué es de la cátedra y qué agregué yo) . . . 4
- 1.3. 0.3 Mapa de la unidad . . . 5
- 1.4. 0.4 El hilo conductor (la historia de la unidad en un párrafo) . . . 5
- 1.5. 0.5 Sugerencia de estudio . . . 6

2. 1. ¿Qué es la calidad? **7**
- 2.1. 1.1 La definición oficial (ISO 9000:2015) . . . 7
- 2.2. 1.2 La calidad es relativa: ¿"calidad" para quién? . . . 7
- 2.3. 1.3 La frase de Crosby: "La calidad no cuesta" . . . 8
- 2.4. 1.4 Para recordar . . . 8

3. 2. Calidad de software y sus cinco perspectivas **9**
- 3.1. 2.1 La definición de calidad de software (IEEE) . . . 9
- 3.2. 2.2 El verdadero desafío: lo que se pide no siempre es lo que se necesita . . . 9
- 3.3. 2.3 Las cinco perspectivas . . . 10
  - 3.3.1. Perspectiva de producto . . . 10
  - 3.3.2. Perspectiva de proceso . . . 10
  - 3.3.3. Perspectiva de las personas usuarias . . . 10
  - 3.3.4. Perspectiva basada en valor . . . 10
  - 3.3.5. Perspectiva trascendental . . . 11
  - 3.3.6. Cuadro comparativo (para memorizar) . . . 11
- 3.4. 2.4 El multiverso de la calidad: un software, cinco miradas . . . 12
- 3.5. 2.5 Para recordar . . . 13

4. 3. Calidad del producto, calidad del proceso, QA y QC **14**
- 4.1. 3.1 Dos caras de la misma moneda . . . 14
- 4.2. 3.2 El proceso influye, pero no garantiza . . . 14
- 4.3. 3.3 QA vs. QC . . . 15
- 4.4. 3.4 Para recordar . . . 16

5. 4. Calidad interna del código y complejidad ciclomática **17**
- 5.1. 4.1 Que funcione no significa que esté bien construido . . . 17
  - 5.1.1. ¿Por qué importa la calidad interna? . . . 17
- 5.2. 4.2 Caso para analizar: los 17 lugares y las 600 líneas . . . 18

5.3. 4.3 Complejidad ciclomática . . . 19
5.3.1. Qué mide . . . 19
5.3.2. Qué cuenta como "decisión" . . . 19
5.3.3. Ejemplo resuelto paso a paso . . . 20
5.3.4. ¿Cuántos casos de prueba hacen falta? . . . 21
5.3.5. Cómo bajar la complejidad: dividir en unidades chicas . . . 21
5.3.6. Umbrales y herramientas . . . 22
5.4. 4.4 Para recordar . . . 22

6. 5. Deuda técnica **23**
6.1. 5.1 La metáfora: "lo que no se corrige hoy se paga mañana... con intereses" . . . 23
6.2. 5.2 No toda deuda es igual: el cuadrante . . . 23
6.3. 5.3 Costos visibles e invisibles de la no calidad: el iceberg . . . 24
6.4. 5.4 Cómo se ve la deuda en las herramientas . . . 25
6.5. 5.5 Cómo gestionar la deuda: hacerla visible . . . 26
6.6. 5.6 Para recordar . . . 27

7. 6. Cumplir el proceso no garantiza la calidad (Aspecto 03) **28**
7.1. 6.1 La tesis . . . 28
7.2. 6.2 Escenario: el "semáforo verde" . . . 28
7.3. 6.3 Escenario: "La empresa certificada" . . . 29
7.4. 6.4 Qué evidencia pedir: proceso vs. producto . . . 30
7.5. 6.5 Para recordar . . . 31

8. 7. Costos de la calidad y de la no calidad **32**
8.1. 7.1 Qué son y para qué sirven . . . 32
8.2. 7.2 Las cuatro categorías . . . 32
8.2.1. Regla práctica para clasificar un costo . . . 33
8.3. 7.3 Conformidad y no conformidad . . . 33
8.4. 7.4 Caso real: CrowdStrike (2024) . . . 34
8.5. 7.5 Cómo medir el costo de la no calidad: COPQ . . . 34
8.5.1. Ejemplo de la slide: plataforma SaaS (período anual) . . . 35
8.6. 7.6 Indicadores económicos de la calidad . . . 35
8.7. 7.7 Caso práctico: Proyecto Ágora . . . 36
8.8. 7.8 Para recordar . . . 38

9. 8. Calidad Total y los ocho principios **39**
9.1. 8.1 ¿Alcanza con que el software funcione? . . . 39
9.2. 8.2 Definición de Calidad Total . . . 39
9.3. 8.3 Los ocho principios, de un vistazo . . . 40
9.4. 8.4 Principio 1 — Orientación al cliente . . . 41
9.5. 8.5 Principio 2 — Liderazgo y compromiso con la calidad . . . 42
9.6. 8.6 Principio 3 — Participación del personal . . . 43
9.7. 8.7 Principio 4 — Enfoque basado en procesos . . . 44
9.8. 8.8 Principio 5 — Enfoque de sistema / Implementación de un SGC . . . 45
9.9. 8.9 Principio 6 — Mejora continua . . . 46
9.9.1. Cuatro modelos de mejora continua . . . 46
9.9.2. Six Sigma / DMAIC aplicado al software: "¿por qué llegan fallas a producción?" 47
9.10. 8.10 Principio 7 — Toma de decisiones basada en evidencia . . . 48
9.10.1. Diagrama de Ishikawa . . . 49
9.11. 8.11 Principio 8 — Relación con los proveedores . . . 50
9.12. 8.12 Cómo se conectan los ocho principios . . . 51

9.13. 8.13 Cómo detectar qué principio se incumple en un caso . . . 51
9.14. 8.14 Para recordar . . . 52

10. 9. Integración: cómo se conectan los temas **53**
10.1. 9.1 Mapa conceptual de la unidad . . . 53
10.2. 9.2 Tabla de reflejos: "si te dicen X, pensá en Y" . . . 54
10.3. 9.3 Un mismo caso, ocho lecturas . . . 55
10.4. 9.4 Relaciones entre temas que conviene tener claras . . . 56
10.5. 9.5 Tres lentes sobre una misma actividad . . . 56

11. 10. Autoevaluación **58**
11.1. 10.1 Cómo usarla . . . 58
11.2. 10.2 Preguntas de desarrollo . . . 58
11.3. 10.3 Opción múltiple . . . 59
11.4. 10.4 Verdadero o falso (justificá) . . . 63
11.5. 10.5 Ejercicios numéricos de costos . . . 65
11.6. 10.6 Ejercicios de complejidad ciclomática . . . 66
11.7. 10.7 Casos para analizar . . . 67

12. 11. Errores y trampas frecuentes **69**
12.1. 11.1 Confusiones clásicas . . . 69
12.2. 11.2 Trampas de redacción en preguntas de opción múltiple . . . 71
12.3. 11.3 Cómo responder un caso en el parcial . . . 71

13. Anexo A. Respuestas de la autoevaluación **72**
13.1. A.1 Preguntas de desarrollo . . . 72
13.2. A.2 Opción múltiple . . . 76
13.3. A.3 Verdadero o falso . . . 78
13.4. A.4 Ejercicios de costos . . . 79
13.4.1. Ejercicio 1 . . . 79
13.4.2. Ejercicio 2 . . . 79
13.4.3. Ejercicio 3 . . . 80
13.4.4. Ejercicio 4 . . . 80
13.5. A.5 Ejercicios de complejidad ciclomática . . . 81
13.5.1. Ejercicio 5: `clasificarPedido` . . . 81
13.5.2. Ejercicio 6: `costo_envio` . . . 82
13.6. A.6 Casos para analizar . . . 83
13.6.1. Caso A: "Funciona, no lo toquemos" . . . 83
13.6.2. Caso B: "Quality Gate: Passed" . . . 83
13.6.3. Caso C: "Retrospectivas sin acciones" . . . 84

14. Anexo B. Glosario **85**

15. Anexo C. Resumen y fórmulas **88**
15.1. C.1 Resumen . . . 88
15.2. C.2 Fórmulas . . . 88
15.3. C.3 Frases para llevarse . . . 89

16. Anexo D. Qué preguntó el parcial 2025 de esta unidad **90**
16.1. D.1 Preguntas de teoría que son de la Unidad I . . . 90
16.2. D.2 Preguntas del mismo examen que **no** son de esta unidad . . . 92
16.3. D.3 Qué conclusión sacar . . . 92



1. 0. Cómo usar este apunte

1.1. 0.1 Qué cubre

Este apunte integra, explica y completa las cuatro presentaciones de la Unidad I 🤘:

1. **Introducción a la Calidad de Software** (definiciones y las cinco perspectivas).
2. **Aspectos de la Calidad de Software** (producto vs. proceso, calidad interna, complejidad ciclomática, deuda técnica, QA vs. QC).
3. **Costos de la Calidad / de la No Calidad** (prevención, evaluación, fallas, COPQ e indicadores).
4. **Principios de la Calidad Total** (definición y los ocho principios).

No es un resumen de las slides: busca que *entiendas* cada tema, por eso incluye explicaciones, ejemplos resueltos, tablas comparativas, casos con respuesta modelo y una autoevaluación con respuestas al final.

1.2. 0.2 Convenciones (cómo distinguir qué es de la cátedra y qué agregué yo)

> **Idea clave.** Lo esencial de cada tema; si solo podés recordar una cosa, que sea esta.

> **Ojo.** Trampa o confusión frecuente.

> **Para el examen.** Cómo suele aparecer el tema o cómo conviene responderlo.

> **Extra (no está en las slides).** Agregado mío para entender mejor o profundizar. Sirve para comprender, pero si el examen dice "según la cátedra", priorizá siempre la formulación de las slides.

> **Respuesta modelo.** Mi análisis de un caso. La cátedra puede esperar matices distintos, pero la lógica de razonamiento es la que importa.

Lo que sale de las presentaciones va sin marca; lo que agregué va señalado como *Extra*, *Respuesta modelo* o *(aporte mío)*. Las aclaraciones menores de redacción, hechas solo para que se entienda mejor, no se marcan. Las cifras de los ejemplos resueltos fueron recalculadas, y las de complejidad ciclomática verificadas con una herramienta real (ESLint).

1.3. 0.3 Mapa de la unidad

Cap.	Tema	Presentación de origen	Tenés que poder…
1	Qué es la calidad	Introducción	Definir calidad (ISO 9000), explicar por qué es relativa y la frase de Crosby
2	Calidad de software y sus 5 perspectivas	Introducción	Definir con IEEE, nombrar las 5 perspectivas con su pregunta clave y clasificar situaciones
3	Producto vs. proceso; QA vs. QC	Aspectos	Diferenciarlos, explicar por qué el proceso "influye pero no garantiza"
4	Calidad interna y complejidad ciclomática	Aspectos	Explicar por qué "que funcione" no alcanza y calcular la complejidad de una función
5	Deuda técnica	Aspectos	Definirla, usar la metáfora del interés, ubicar casos en el cuadrante y gestionarla
6	Cumplir el proceso no garantiza la calidad	Aspectos	Analizar los casos "semáforo verde" y "empresa certificada"
7	Costos de la calidad	Costos	Clasificar costos, calcular COPQ e indicadores, interpretar el caso Ágora
8	Calidad Total y 8 principios	Principios	Definir calidad total, explicar cada principio, detectar cuál se incumple en un caso
9	Integración	todas	Conectar los temas entre sí
10	Autoevaluación	—	Practicar (respuestas en el Anexo A)
11	Errores frecuentes	—	Evitar las confusiones clásicas

1.4. 0.4 El hilo conductor (la historia de la unidad en un párrafo)

La calidad no es una cosa única: depende de *para quién* y *respecto de qué requisitos* se mide (cap. 1), y en software se puede mirar desde cinco perspectivas distintas (cap. 2). Dos de esas miradas son especialmente importantes: la calidad del **producto** y la del **proceso** (cap. 3). Un producto puede

funcionar y aun así estar mal construido por dentro (cap. 4), y esa mala construcción se acumula como **deuda técnica** que se paga con intereses (cap. 5). Además, un proceso impecable no garantiza un buen producto (cap. 6). Todo esto se traduce en **plata**, y se puede medir (cap. 7). Por último, la **Calidad Total** propone una forma de gestionar la organización entera para que la calidad no dependa de la suerte ni de esfuerzos aislados (cap. 8).

1.5. 0.5 Sugerencia de estudio

1. **Primera pasada:** leé los capítulos 1 a 8 con calma y hacé los cuadros a mano al menos una vez (perspectivas, QA/QC, cuadrante de deuda, cuatro costos, ocho principios).
2. **Segunda pasada:** mirá solo los recuadros *Idea clave*, *Ojo* y *Para el examen*, y el Anexo C (resumen y fórmulas).
3. **Tercera pasada:** resolvé la autoevaluación (cap. 10) *sin mirar* el Anexo A y recién después corregí.

2. 1. ¿Qué es la calidad?

2.1. 1.1 La definición oficial (ISO 9000:2015)

> **Calidad** es el *grado* en el que un conjunto de *características inherentes* de un *objeto* cumple con los *requisitos*. — ISO 9000:2015

Parece una frase abstracta, pero cada palabra está elegida con cuidado. Desarmémosla:

Palabra	Qué significa	Ejemplo en software
Objeto	Lo que se está evaluando: un producto, un servicio o un proceso	Una app de turnos médicos; el proceso de despliegue
Características inherentes	Lo que el objeto *es y hace*: propiedades que existen en él	Tiempo de respuesta, facilidad de uso, seguridad
Requisitos	Necesidades, expectativas y obligaciones	Necesidad: "ver mis turnos"; expectativa: "que sea simple"; obligación: "cumplir la ley de protección de datos"
Grado	Medida de cumplimiento, no un sí/no	"Cumple casi todo" no es lo mismo que "cumple poco"

De esta definición se desprenden tres ideas:

1. **La calidad es una relación**, no una propiedad aislada. No existe "calidad" del objeto por sí solo: existe calidad *del objeto respecto de ciertos requisitos*.
2. **Es una cuestión de grado.** Hay más o menos calidad; no se reduce a "tiene calidad / no tiene calidad".
3. **Los requisitos no son solo lo que está escrito.** Incluyen necesidades y expectativas que a veces nadie escribió (por ejemplo, "que no se cuelgue").

> **Extra (no está en las slides).** ISO distingue las características *inherentes* (existen en el objeto de manera permanente, como el rendimiento de un sistema) de las *asignadas* (se le atribuyen desde afuera, como el precio o quién es su propietario). La calidad se evalúa sobre las inherentes.

2.2. 1.2 La calidad es relativa: ¿"calidad" para quién?

La presentación lo ilustra con dos hamburguesas:

- Una doble con cheddar, panceta y papas fritas: *"para mí: abundante y sabrosa"*.
- Una vegetal con palta, tomate y ensalada: *"para mí: fresca y saludable"*.

La conclusión de la slide: **la calidad depende de las necesidades, expectativas y requisitos de cada persona. Un mismo producto puede ser excelente para alguien e inadecuado para otra persona.**

¿Qué significa esto para el trabajo en software?

- Antes de juzgar si algo "es de calidad", hay que preguntar **respecto de qué requisitos y de quién** (usuario final, cliente que paga, equipo que mantiene, organización).
- Por eso la **ingeniería de requisitos** es tan importante: si los requisitos no están claros, no hay contra qué medir.
- *Relativa* no quiere decir *arbitraria*: una vez que los requisitos están explícitos, se puede medir el grado de cumplimiento. Hacer explícito lo implícito es justamente parte del trabajo.

> **Ojo.** Cuando en el examen aparezca "la calidad es subjetiva", la idea no es que "cada uno opina lo que quiere", sino que **los requisitos cambian según quién los plantea**.

2.3. 1.3 La frase de Crosby: "La calidad no cuesta"

> *"La calidad no cuesta. No es un regalo, pero es gratuita. Lo que cuesta dinero son las cosas que no tienen calidad: todas las acciones que resultan de no hacer bien las cosas la primera vez."* — Philip B. Crosby

Cómo leerla:

- **No dice que prevenir sea gratis** ("no es un regalo": requiere esfuerzo e inversión).
- Dice que **lo realmente caro es la no calidad**: retrabajo, correcciones, reclamos, soporte, clientes perdidos.
- Por eso invertir en hacerlo bien desde el principio *se paga solo*: lo que se gasta en prevenir es menos de lo que se evita gastar en corregir.

Esta idea es el puente directo al capítulo 7 (costos de la calidad y de la no calidad) y al capítulo 5 (deuda técnica: "lo que no se corrige hoy se paga mañana con intereses").

> **Extra (no está en las slides).** La frase proviene del libro *Quality Is Free* (1979). Crosby asociaba esta idea con el objetivo de "cero defectos" y con "hacerlo bien a la primera".

2.4. 1.4 Para recordar

- Calidad (ISO 9000:2015) = **grado** en que las **características inherentes** de un **objeto** cumplen los **requisitos** (necesidades, expectativas, obligaciones).
- La calidad es **relativa a los requisitos de quien la evalúa** (hamburguesas).
- Crosby: **lo que cuesta es la no calidad**, no la calidad.

3. 2. Calidad de software y sus cinco perspectivas

3.1. 2.1 La definición de calidad de software (IEEE)

> La calidad del software es el **grado** con el que un **sistema, componente o proceso** cumple los **requerimientos especificados** y las **necesidades o expectativas** del cliente o usuario. — IEEE

Comparada con la de ISO:

	ISO 9000:2015 (general)	IEEE (software)
Qué evalúa	Cualquier objeto (producto, servicio, proceso)	Un sistema, un componente o un proceso
Contra qué	Los requisitos (incluyen necesidades, expectativas, obligaciones)	Dos cosas separadas: requerimientos **especificados** y necesidades o expectativas del cliente/usuario
Esqueleto común	"Grado" de cumplimiento	"Grado" de cumplimiento

La definición de IEEE hace explícito algo que en ISO está implícito: **cumplir lo escrito no alcanza**; también hay que responder a lo que la persona realmente necesita y espera.

3.2. 2.2 El verdadero desafío: lo que se pide no siempre es lo que se necesita

La slide lo plantea con dos globos de diálogo:

- **Lo que se solicita:** los requerimientos expresados, lo que la persona *dice* que quiere.
- **Lo que realmente se necesita:** las necesidades y expectativas, lo que la persona espera obtener *incluso cuando no logra expresarlo con claridad.*

Y concluye: **un software de calidad no solo cumple lo especificado; también responde a las necesidades y expectativas de quienes lo utilizan.**

> **Extra (ejemplo propio).** Un área comercial pide "un listado de ventas exportable a Excel". El sistema lo entrega perfecto (cumple lo especificado). Pero lo que el área necesitaba era *decidir qué mercadería reponer*, y para eso el listado no sirve. El software cumplió el requerimiento y falló la necesidad. Esta tensión es exactamente la que más adelante en la carrera vas a ver como diferencia entre **verificación** (¿construimos bien lo especificado?) y **validación** (¿construimos lo que realmente se necesitaba?).

3.3. 2.3 Las cinco perspectivas

La presentación insiste: **la calidad del software no tiene una única mirada.** Hay cinco perspectivas, y cada una responde una pregunta distinta.

3.3.1. Perspectiva de producto

- **Idea:** la calidad está en las **características medibles** del software.
- **Pregunta clave:** ¿qué propiedades tiene el producto?
- **Ejemplos (de la slide):** rendimiento, complejidad, mantenibilidad, defectos.
- **Cómo se evalúa (aporte mío):** con métricas, análisis estático, pruebas.
- **Límite (aporte mío):** un producto puede tener muy buenas métricas y aun así no servirle a nadie.

3.3.2. Perspectiva de proceso

- **Idea:** la calidad está en **desarrollar el software de acuerdo con procesos y especificaciones definidos**.
- **Pregunta clave:** ¿se construyó correctamente?
- **Ejemplos (de la slide):** estándares, revisiones, pruebas, trazabilidad.
- **Límite (la cátedra lo desarrolla en el Aspecto 03):** un proceso impecable no garantiza un buen producto (ver capítulo 6).

3.3.3. Perspectiva de las personas usuarias

- **Idea:** la calidad depende de que el software **satisfaga las necesidades de quienes lo utilizan en su contexto real**.
- **Pregunta clave:** ¿le sirve realmente a la persona que lo usa?
- **Ejemplos (de la slide):** facilidad de uso, accesibilidad, utilidad, satisfacción.
- **Límite (aporte mío):** las necesidades de distintos usuarios pueden contradecirse y hay restricciones de costo y tiempo.

3.3.4. Perspectiva basada en valor

- **Idea:** la calidad surge del **equilibrio entre los beneficios** que aporta el software, **su costo** y **las restricciones del proyecto**.
- **Pregunta clave:** ¿qué nivel de calidad aporta mayor valor?
- **Ejemplos (de la slide):** funcionalidades prioritarias, costo, tiempo, beneficio para las personas usuarias.

- **Cómo lo plantea la slide:** el mayor valor se logra cuando el software entrega el máximo beneficio posible, cumpliendo las restricciones de costo y tiempo del proyecto y enfocándose en lo que realmente importa a las personas usuarias.
- **Interpretación mía:** no se busca "la máxima calidad en todo", sino la **calidad suficiente donde más valor genera**.

3.3.5. Perspectiva trascendental

- **Idea:** la calidad se reconoce como **excelencia**: se percibe en la experiencia global, aunque resulte difícil definirla por completo.
- **Pregunta clave:** ¿qué hace que percibamos un software como excelente?
- **Ejemplos (de la slide):** experiencia cuidada, consistencia, confianza, innovación.
- **Interpretación mía:** es la calidad que "se nota cuando la ves, pero cuesta decir exactamente qué es".

3.3.6. Cuadro comparativo (para memorizar)

Perspectiva	Frase que la resume	Pregunta clave	Ejemplos que menciona la cátedra
Producto	Calidad en las características **medibles**	¿Qué propiedades tiene el producto?	Rendimiento, complejidad, mantenibilidad, defectos
Proceso	Calidad en **cómo** se desarrolla	¿Se construyó correctamente?	Estándares, revisiones, pruebas, trazabilidad
Personas usuarias	Calidad = satisface a quien lo usa en su contexto	¿Le sirve realmente a quien lo usa?	Facilidad de uso, accesibilidad, utilidad, satisfacción
Valor	Equilibrio beneficio / costo / restricciones	¿Qué nivel de calidad aporta mayor valor?	Funcionalidades prioritarias, costo, tiempo, beneficio
Trascendental	Excelencia percibida, difícil de definir	¿Qué hace que lo percibamos como excelente?	Experiencia cuidada, consistencia, confianza, innovación

> **Para el examen.** El parcial 2025 preguntó cuál afirmación describe la perspectiva "basada en el producto". La respuesta correcta era *"la calidad se relaciona con características inherentes y medibles del software, como la complejidad ciclomática o el número de líneas de código"*. Las otras opciones eran, en realidad, las otras perspectivas: *conformidad con procesos y estándares* (proceso), *satisface necesidades y expectativas del cliente* (usuario) y *excelencia que se reconoce pero es difícil de definir* (trascendental). Una buena técnica: **asociá cada perspectiva con su palabra ancla**: producto = *medible*; proceso = *conformidad con procesos*; personas = *necesidades del usuario*; valor = *equilibrio costo-beneficio*; trascendental = *excelencia difícil de definir*.

> **Extra (no está en las slides).** Estas cinco perspectivas vienen de los "cinco enfoques de la calidad" de David Garvin (1984), que luego Kitchenham y Pfleeger (1996) aplicaron al software. En la bibliografía las vas a encontrar con otros nombres: *trascendente*, *basada en el producto*, *basada en el usuario*, *basada en la fabricación* (equivale a "proceso") y *basada en el valor*. Por eso el examen puede decir "perspectiva basada en el producto" o "basada en el usuario": son las mismas cinco, con el nombre de la bibliografía.

3.4. 2.4 El multiverso de la calidad: un software, cinco miradas

La presentación cierra con una consigna: *"Un mismo software. Cinco miradas. ¿Una sola calidad?"* (la actividad grupal consiste en representar, con personajes como usuario, desarrollador, líder de proyecto, QA y negocio, un conflicto real sobre si "este software es de calidad… ¿o no?").

La respuesta de fondo es: **no hay una sola calidad; hay una calidad integrada que equilibra cinco miradas, y las miradas pueden entrar en conflicto.**

Personaje	Qué le importa	Perspectiva predominante (tendencia, no regla)
Usuaria	Que sea simple, rápida y útil	Personas usuarias
Desarrollador	Que el código sea claro y modificable	Producto (calidad interna)
QA / testing	Que se verifique y no haya defectos	Producto y proceso
Líder de proyecto	Cumplir plazos, costos y procesos	Proceso (y valor)
Negocio	Retorno, diferenciación, ventas	Valor

Ejemplos de conflicto:

- La usuaria quiere una nueva función ya; el desarrollador necesita tiempo para refactorizar (sin cambios visibles) para que esa función no rompa todo.
- El líder de proyecto quiere cerrar el sprint; QA pide más pruebas.
- Negocio quiere lanzar antes que la competencia; el equipo advierte sobre seguridad.

> **Idea clave.** Los conflictos no se resuelven diciendo "quién tiene razón", sino **haciendo explícitos los requisitos y las prioridades**. La perspectiva de **valor** es la que sirve de árbitro: pregunta qué nivel de calidad conviene en cada cosa, dados costo, tiempo y beneficio.

> **Extra (ejemplo para fijar las cinco, con una app de homebanking).** *Producto:* el tiempo de respuesta en transferencias es menor a 2 segundos y no hay defectos críticos abiertos. *Proceso:* cada cambio pasa por revisión de código, pruebas automáticas y trazabilidad al requerimiento. *Personas:* una persona mayor puede transferir sola desde el celular y con lector de pantalla. *Valor:* se prioriza seguridad y confiabilidad de las transferencias sobre animaciones vistosas. *Trascendental:* la app "se siente sólida": todo es coherente, genera confianza y nunca falla en el momento importante.

3.5. 2.5 Para recordar

- **IEEE:** calidad de software = grado en que cumple los **requerimientos especificados** *y* las **necesidades o expectativas** del usuario.
- **Desafío central:** lo que se pide $\neq$ lo que se necesita.
- **Cinco perspectivas:** producto (¿qué propiedades?), proceso (¿se construyó correctamente?), personas (¿le sirve a quien lo usa?), valor (¿qué nivel aporta más valor?), trascendental (¿qué lo hace excelente?).
- **No hay una sola calidad:** hay una calidad integrada que equilibra cinco miradas, y la de valor ayuda a decidir.

4. 3. Calidad del producto, calidad del proceso, QA y QC

4.1. 3.1 Dos caras de la misma moneda

La segunda presentación abre con un título directo: **la calidad del software integra la calidad del producto y la calidad del proceso.**

	Calidad del producto	Calidad del proceso
Qué es	Grado en que el producto digital cumple los requisitos, satisface las necesidades de las personas usuarias y permite alcanzar los resultados esperados	Grado en que las actividades, métodos y prácticas para desarrollar y mantener el software están definidos, se aplican de manera consistente, se controlan y mejoran
Dónde se observa	En lo que el software hace y en la experiencia que ofrece	En cómo se planifica, desarrolla, prueba, entrega y mejora el software
Qué percibe / se pregunta	Lo que la persona usuaria percibe. ¿Cumple su propósito?	Cómo se organiza y controla el desarrollo. ¿Es repetible, medible y mejorable?
Ejemplos	En la slide: una app donde el pago se realiza con éxito y el resumen (monto, fecha, estado) se muestra correctamente	Prácticas y estándares definidos, revisiones y controles, pruebas integradas al proceso, gestión de cambios y riesgos, medición y mejora continua

> **Ojo (dos usos de la palabra "producto").** En la presentación de *Introducción*, la "perspectiva de producto" se centra en las **características medibles** (complejidad, defectos, rendimiento). En *Aspectos*, la "calidad del producto" es más amplia: incluye si el producto cumple requisitos y *satisface a las personas usuarias*. Una forma de reconciliarlo (interpretación mía): "calidad del producto" es el **resultado completo** que se entrega y se puede evaluar desde varias perspectivas, mientras que la "perspectiva de producto" es **una** de esas lentes (la de las propiedades medibles). Si preguntan "¿qué describe la perspectiva basada en el producto?", respondé con la de *Introducción* (propiedades medibles).

4.2. 3.2 El proceso influye, pero no garantiza

La idea más importante de este capítulo está en la slide "Proceso y producto: dos miradas de la calidad":

> **El proceso crea las condiciones. El producto confirma la calidad.**

- **Calidad del proceso:** buenas prácticas, controles y mejora continua **reducen la variabilidad y previenen defectos**.
- **Calidad del producto:** pruebas y métricas **verifican si el software es confiable, seguro, usable y mantenible**.
- Entre ambos hay una relación de **"influye, no garantiza"**:
  - Un mal proceso **aumenta el riesgo** de errores, retrabajo y resultados inconsistentes.
  - Un buen proceso **reduce** ese riesgo, pero **no asegura** por sí solo un buen producto.

> **Extra (analogía propia).** Seguir una receta al pie de la letra (proceso) hace mucho más probable que el plato salga bien, pero la única forma de saber si salió bien es **probarlo** (producto). Y a la inversa: acertar por casualidad, sin repetibilidad, no es sostenible.

La frase de cierre del Aspecto 03 lo resume: **cumplir el proceso ≠ garantizar la calidad** (se analiza con casos en el capítulo 6).

4.3. 3.3 QA vs. QC

Una de las confusiones más frecuentes es mezclar *Quality Assurance* (QA) con *Quality Control* (QC). La cátedra lo resume así: **prevenir en el proceso, verificar en el producto.**

	Quality Assurance (QA) — *previene*	**Quality Control (QC)** — *verifica*
Pregunta principal	¿Cómo evitamos que aparezcan defectos?	¿El producto tiene defectos?
Foco	Proceso y prácticas de desarrollo	Producto de software
Orientación	Preventiva	Detectiva / verificadora
Ejemplos actuales	*Definition of Done*, estándares, code review, quality gates, estrategia de testing, CI/CD	Pruebas funcionales, de regresión, automatización, performance, seguridad, pruebas exploratorias
Perfiles	Quality Engineer, QA Lead, SDET, Quality Manager	Software Tester, QC Analyst, Test Automation Engineer, Performance Tester

Y dos frases para llevarse:

- **QA construye las condiciones para la calidad; QC aporta evidencia sobre la calidad del producto.**
- **QA no es sinónimo de testing.** En *Quality Engineering*, la calidad es **responsabilidad de todo el equipo**.

> **Ojo.** Que alguien tenga el título "QA" no significa que su tarea sea "QA" en el sentido del cuadro. Un "QA" que ejecuta casos de prueba manuales está haciendo, conceptualmente, **QC**. Y si el cargo es "Tester", pero define la estrategia de pruebas y los criterios de aceptación, está haciendo **QA**.

> **Extra.** El límite entre QA y QC no es nítido: un code review previene defectos futuros (QA) pero también encuentra defectos en el código (QC). Lo que decide es el **foco**. Regla práctica: *si la actividad busca que el defecto no aparezca, es QA; si busca encontrar el que ya existe, es QC.*

4.4. 3.4 Para recordar

- La calidad del software **integra** producto y proceso.
- **Proceso crea condiciones; producto confirma la calidad.** Influye, pero no garantiza.
- **QA = prevenir (proceso); QC = verificar (producto).** QA $\neq$ testing. La calidad es de todo el equipo.

5. 4. Calidad interna del código y complejidad ciclomática

5.1. 4.1 Que funcione no significa que esté bien construido

El *Aspecto 01* de la presentación muestra un auto deportivo que arranca y muestra "ÉXITO" en el tablero… pero con el chasis abierto y un enjambre de cables enredados por dentro. La pregunta que deja la slide es la importante: **¿qué pasaría cuando haya que modificarlo?**

Un software puede:

- dar los resultados esperados (**calidad externa** aceptable: el usuario no ve problemas), y
- estar construido de una manera tan confusa que **cada cambio sea lento, riesgoso y caro** (**calidad interna** deficiente).

> **Idea clave.** Un software **no es de calidad solo porque produce el resultado esperado**. Su código debe ser **claro, modular, comprensible y fácil de modificar**.

5.1.1. ¿Por qué importa la calidad interna?

La slide da cuatro razones, y conviene entender la lógica de cada una:

Razón	Por qué
Reduce errores al cambiarlo	Si el código es claro y modular, un cambio en un lugar no rompe otros
Facilita el mantenimiento	Es más fácil entender qué hace cada parte y corregirla
Disminuye tiempos y costos	Cada cambio requiere menos esfuerzo
Permite que el software evolucione	El software vive años: los requisitos cambian y hay que poder seguirlos

> **Ojo (por qué el usuario "no ve" este problema).** Mientras nadie toque el código, la mala calidad interna es invisible. Se manifiesta cuando hay que *cambiar*: entonces aparecen los retrasos, los errores nuevos y los "esto era para unas horas" que terminan siendo días. Por eso es una trampa decir "funciona perfecto, ¿cuál es el problema?".

5.2. 4.2 Caso para analizar: los 17 lugares y las 600 líneas

Escenario (resumen de la slide). Una empresa de comercio electrónico usa desde hace cuatro años un sistema propio de gestión de pedidos. Procesa más de **8.000 pedidos por día** y los clientes compran, pagan, reciben confirmación y consultan el envío sin inconvenientes. El gerente dice: *"No entiendo por qué el equipo de desarrollo insiste en que tenemos problemas con el sistema. Los clientes compran todos los días y funciona perfectamente."*

La empresa lanza una campaña y pide un cambio "simple": **envío gratis para clientes premium cuando la compra supera cierto monto**. El equipo estima unas pocas horas. Al empezar descubre que:

- la lógica de cálculo del envío está **repetida en 17 lugares** distintos;
- hay un **método de más de 600 líneas** que mezcla pedidos, descuentos, stock, envío y facturación;
- modificar una regla de descuentos **genera errores en otras partes**;
- **casi no existen pruebas automatizadas**;
- distintos programadores implementaron **versiones diferentes del mismo cálculo**;
- hay sectores que **nadie quiere tocar** por miedo a romper algo;
- un cambio estimado en horas termina demandando **seis días** y varias correcciones posteriores.

Preguntas de la slide y respuesta modelo:

> **Respuesta modelo.**
>
> 1. *¿Dirían que es un software de calidad?* Depende de la perspectiva. Para el usuario hoy funciona (calidad externa aceptable), pero su **calidad interna es baja**, y eso lo vuelve costoso de cambiar. Con la definición de calidad como cumplimiento de requisitos, el requisito de "poder evolucionar" (mantenibilidad) **no se cumple**. Conclusión: calidad **insuficiente**, con riesgo creciente.
> 2. *Si funciona, ¿dónde está el problema?* En **cómo está construido**, no en lo que hace. El problema se hace visible recién cuando hay que modificarlo.
> 3. *¿Qué riesgos aparecen al modificarlo?* Regresiones (arreglar algo rompe otra cosa), **inconsistencias** (olvidarse una de las 17 copias o que cada copia haga algo distinto), estimaciones erróneas, miedo a tocar partes del sistema, entregas lentas y errores en producción por falta de pruebas.
> 4. *¿Qué características de su construcción interna llaman la atención?* **Duplicación** de lógica (17 lugares), un método enorme con **demasiadas responsabilidades** (600 líneas que mezclan cinco temas), **alto acoplamiento** (un cambio en descuentos afecta otras partes), **falta de pruebas automatizadas** e implementaciones inconsistentes del mismo cálculo.
> 5. *¿Qué frase resume el caso?* "**Que funcione no significa que esté bien construido.**" (o: "lo que no se corrige hoy se paga mañana con intereses").

> **Extra.** Un método de 600 líneas que mezcla pedidos, descuentos, stock, envío y facturación casi seguro tiene una **complejidad ciclomática muy alta** (es una inferencia mía, el enunciado no da el número). Y el caso es un ejemplo perfecto de **deuda técnica**: la diferencia entre "unas horas" y "seis días" es el **interés** que se está pagando (capítulo 5).

5.3. 4.3 Complejidad ciclomática

5.3.1. Qué mide

La **complejidad ciclomática** es una métrica que **estima la cantidad de caminos linealmente independientes** que existen en el código. Dicho más simple: *cuántos recorridos distintos puede seguir la ejecución según las decisiones que toma el programa.*

La slide da una regla práctica para interpretarla:

$$\text{Complejidad} \approx \text{decisiones} + 1$$

y una consecuencia: **a mayor complejidad, más difícil es comprender, probar, modificar y mantener el software.**

5.3.2. Qué cuenta como "decisión"

Construcción (estilo JavaScript/ESLint)	Aporta
Base (toda función arranca con un camino)	+1
`if` y cada `else if`	+1 cada uno
`else`	0 (no es una decisión nueva: es la salida de la anterior)
`for`, `while`, `do ... while`, `for ... of`	+1 cada uno
Cada `case` de un `switch`	+1 cada uno (el `default` no suma)
Operadores lógicos de una condición compuesta (Y, O: en JavaScript `&&` y el "o" lógico)	+1 por operador (ver *Ojo* abajo)
Operador ternario `? :`	+1
`catch`	+1

> **Ojo (condiciones compuestas).** Algunas herramientas (como ESLint) cuentan cada `&&` / `||` como una decisión adicional; algunos textos clásicos cuentan la condición compuesta como **una sola** decisión. El resultado puede diferir en uno o más. **En un examen, aclará qué convención usás.** Es lo mismo que pasa en el parcial 2025 con la función `costo_envio`: da complejidad 6 si el `and` se cuenta aparte y 5 si se cuenta como una sola decisión.

5.3.3. Ejemplo resuelto paso a paso

Función (inspirada en la captura de ESLint de la slide, que marca "complexity of 8, maximum allowed is 5"; el código completo de la slide no se ve, así que armé uno propio con el mismo resultado):

```
function calcularDescuento(monto, cliente, cupon) {
  let descuento = 0;
  if (monto > 10000) {                          // +1
    descuento += 5;
  }
  if (cliente.esPremium &&                      // +1 (if) +1 (operador Y)
      cliente.antiguedad > 2) {
    descuento += 10;
  }
  for (const item of cliente.carrito) {         // +1
    if (item.enOferta) {                        // +1
      descuento += 1;
    }
  }
  switch (cupon) {
    case "BIENVENIDA":                          // +1
      descuento += 10;
      break;
    case "FRECUENTE":                           // +1
      descuento += 5;
      break;
    default:                                    // 0
      break;
  }
  return descuento;
}
```

Cuenta: decisiones = `if` (1) + `if` (1) + `&&` (1) + `for` (1) + `if` (1) + `case` (1) + `case` (1) = **7**. Complejidad = 7 + 1 = **8**. Esto lo verifiqué ejecutando ESLint con la regla `complexity`: devuelve exactamente 8. Con un máximo permitido de 5, la herramienta marcaría un error, igual que en la slide.

5.3.4. ¿Cuántos casos de prueba hacen falta?

Esa complejidad **8** significa que hay 8 caminos linealmente independientes, y por lo tanto se necesitan como mínimo **8 casos de prueba** para cubrir los *caminos básicos* (esto es lo que se usa en testing de caja blanca; el parcial 2025 pregunta justamente que la complejidad ciclomática "se utiliza para determinar el número mínimo de caminos independientes a probar"). Una forma de armarlos: partir de un camino base (todas las decisiones en "falso") y cambiar **una decisión por vez**:

N.º	Qué decisión se activa	Entrada (monto, cliente, cupón)	Resultado
1	Camino base (todo "falso")	5000, no premium, carrito vacío, `"X"`	0
2	`monto > 10000`	20000, no premium, vacío, `"X"`	5
3	`if` premium (ambas condiciones verdaderas)	5000, premium con 5 años, vacío, `"X"`	10
4	El `&&`: primera verdadera, segunda falsa	5000, premium con 1 año, vacío, `"X"`	0
5	Entra al `for`, `if` interno falso	5000, no premium, 1 ítem sin oferta, `"X"`	0
6	`if` interno verdadero	5000, no premium, 1 ítem en oferta, `"X"`	1
7	`case "BIENVENIDA"`	5000, no premium, vacío, `"BIENVENIDA"`	10
8	`case "FRECUENTE"`	5000, no premium, vacío, `"FRECUENTE"`	5

Los resultados esperados los comprobé ejecutando la función. (El camino del `default` queda cubierto por el caso 1.)

5.3.5. Cómo bajar la complejidad: dividir en unidades chicas

```
function calcularDescuento(monto, cliente, cupon) {
  return descuentoPorMonto(monto)
       + descuentoPorCliente(cliente)
       + descuentoPorOfertas(cliente.carrito)
       + descuentoPorCupon(cupon);
}
```

Con cuatro funciones auxiliares (una por regla), medidas con la misma herramienta: `descuentoPorMonto` = 2, `descuentoPorCliente` = 3, `descuentoPorOfertas` = 3, `descuentoPorCupon` = 3 y la función que las combina = 1.

> **Ojo (no se evapora la complejidad).** Si sumás: 2 + 3 + 3 + 3 + 1 = 12, más que el 8 original, porque cada función nueva suma su propio "+1". **Dividir no hace desaparecer las decisiones: las reparte.** Lo que mejora es que **cada unidad es pequeña, se entiende de un vistazo y se prueba de forma independiente** (tres o cuatro casos por función). Eso es lo que significa la frase de la slide:

> **Idea clave.** *"El objetivo no es llegar siempre a 1, sino mantener cada unidad de código **simple, testeable y comprensible**."* ¡No al código *spaghetti*!

5.3.6. Umbrales y herramientas

- En la slide, la herramienta (ESLint) está configurada con un máximo de **5**. Es un ejemplo: **no existe un número universal**, el equipo lo define.
- Herramientas online que menciona la cátedra: **SonarQube Cloud** (mide y monitorea la complejidad del repositorio), **Codacy** (detecta métodos y archivos complejos) y **DeepSource** (analiza funciones y permite configurar umbrales). La slide final de la sección invita a practicar con el repositorio de la cátedra (`github.com/noeliapinto/AACSW_CP`).

> **Extra.** McCabe, el autor de la métrica, sugirió 10 como referencia clásica; ESLint trae 20 como valor por defecto. Son convenciones, no leyes.

> **Extra (la fórmula "formal").** En teoría de grafos, la complejidad ciclomática se define como $V(G) = E - N + 2P$, con E aristas, N nodos y P componentes conexos del grafo de flujo de control. Para un programa estructurado con una sola entrada y una sola salida, da lo mismo que "decisiones + 1". Es la que vas a ver en caja blanca cuando dibujen el grafo.

5.4. 4.4 Para recordar

- **Funciona $\neq$ está bien construido.** La calidad interna se nota al **cambiar**.
- Calidad interna: código **claro, modular, comprensible, fácil de modificar** $\rightarrow$ menos errores, más mantenibilidad, menos costos, más evolución.
- **Complejidad ciclomática** $\approx$ decisiones + 1 = caminos linealmente independientes = mínimo de casos para cubrir caminos básicos.
- Meta: unidades **simples, testeables y comprensibles**, no "complejidad 1".

6. 5. Deuda técnica

6.1. 5.1 La metáfora: "lo que no se corrige hoy se paga mañana… con intereses"

El *Aspecto 02* abre con una frase y una pregunta: **"Lo que no se corrige hoy se paga mañana… con intereses."** / **"¿Cuánto cuesta realmente una decisión técnica apresurada?"**

Definición de la cátedra. La **deuda técnica** son **deficiencias acumuladas en la calidad interna del software que hacen más costoso modificarlo, mantenerlo y hacerlo evolucionar.** El esfuerzo adicional que exige cada cambio funciona como el **interés** de una deuda.

Cómo mapear la metáfora (respuesta modelo / aporte mío):

En el mundo financiero	En software
Pedir prestado	Tomar un atajo: copiar y pegar, saltear diseño, no escribir pruebas
Interés	**El esfuerzo extra que hay que hacer cada vez que se toca esa zona** (lo dice la slide)
Pagar la deuda	Refactorizar, escribir pruebas, rediseñar
No pagar	El interés crece y los cambios se vuelven cada vez más caros

El caso de los 17 lugares es un buen ejemplo: pasar de "unas horas" a "seis días" es el interés.

Señales para detectarla (según la cátedra): complejidad alta, duplicación de código y *code smells* (síntomas en el código que sugieren un mal diseño).

> **Ojo (qué no es deuda técnica).** No es "el dinero que se gasta en licencias y herramientas" ni "un bug que hay que arreglar". Es **trabajo futuro adicional** causado por una decisión (consciente o no) que priorizó lo inmediato por sobre la calidad interna. (Este tema apareció en el parcial 2025.)

6.2. 5.2 No toda deuda es igual: el cuadrante

La slide propone clasificar la deuda con dos ejes:

- **Eje vertical:** *excesiva/arriesgada* vs. *moderada/menos arriesgada.*
- **Eje horizontal:** *voluntaria* (se decide a conciencia) vs. *involuntaria* (aparece sin que el equipo lo busque).

	Voluntaria	**Involuntaria**
Excesiva / arriesgada	*"No hay tiempo para diseñar"*	*"¿Qué es arquitectura?"*

	Voluntaria	Involuntaria
Moderada / menos arriesgada	*"Entregamos ahora y planificamos corregir"*	*"Ahora sabemos cómo hacerlo mejor"*

Cómo leer cada cuadrante (interpretación mía):

- **Excesiva + voluntaria — "No hay tiempo para diseñar".** El equipo sabe que el atajo es malo y lo toma igual, sin plan de corrección. Es la más peligrosa: decisión consciente, riesgo alto, sin plan de pago.
- **Excesiva + involuntaria — "¿Qué es arquitectura?".** La deuda aparece por **desconocimiento** de buenas prácticas. Se resuelve con formación, mentoría y revisiones.
- **Moderada + voluntaria — "Entregamos ahora y planificamos corregir".** Decisión consciente, riesgo acotado y con **plan de pago**: puede ser una buena decisión de negocio (lanzar rápido para validar) si efectivamente se paga.
- **Moderada + involuntaria — "Ahora sabemos cómo hacerlo mejor".** Es la deuda "natural": recién al terminar se entiende cuál era el mejor diseño. Es inevitable en parte; se gestiona con refactorización continua.

> **Idea clave (la frase de la slide).** *"No toda deuda es igual: importa **cómo se genera**, **cuánto riesgo** implica y si existe un **plan para saldarla**."*

> **Extra (no está en las slides).** Este cuadrante es el *Technical Debt Quadrant* de Martin Fowler (2009): *imprudente/prudente* × *deliberada/inadvertida* (en inglés: *reckless/prudent* × *deliberate/inadvertent*). La metáfora de "deuda técnica" fue propuesta por Ward Cunningham en 1992.

6.3. 5.3 Costos visibles e invisibles de la no calidad: el iceberg

La slide dibuja un iceberg: **solo se ve la punta.** Lo que se registra en un presupuesto suele ser una pequeña parte del costo real.

Costos visibles (la punta)	**Costos invisibles** (lo que está bajo el agua)
Corrección de defectos	Pérdida de confianza
Retrabajo	Daño reputacional
Soporte e incidentes	Menor productividad
Reclamos y garantías	Desgaste del equipo
Demoras inmediatas	Oportunidades perdidas
	Mayor costo de evolución

Costos visibles (la punta)	**Costos invisibles** (lo que está bajo el agua)
	Decisiones demoradas

La pregunta que deja la slide: **¿qué costos estamos midiendo y cuáles permanecen ocultos?**

> **Para el examen.** Un patrón de pregunta posible: "¿cuál de estos es un costo invisible?" Los invisibles son los **difíciles de medir y de atribuir a un defecto concreto** (confianza, reputación, desgaste, oportunidades). Los visibles son los que aparecen directamente en horas, facturas o tickets (corrección, retrabajo, soporte, garantías).

6.4. 5.4 Cómo se ve la deuda en las herramientas

La presentación muestra capturas de **SonarQube** (herramienta de análisis estático) y un gráfico de acumulación. Para interpretar lo que muestran:

Elemento de SonarQube	Qué es	Lo que se ve en las capturas
Technical Debt	Esfuerzo estimado para corregir los problemas de mantenibilidad (expresado en días)	Un panel muestra 993 días, otro 143 días (son capturas distintas)
Issues	Cantidad de problemas detectados	53 mil
Duplications	Porcentaje de código duplicado	78,8 % y 2,4 mil bloques duplicados
Lines of code	Tamaño del proyecto	406 mil líneas (91,2 % JavaScript, 8,8 % C#)
Reliability / Security / Maintainability	Calificaciones de A (mejor) a E (peor) para bugs, vulnerabilidades y *code smells*	Bugs: 1.349 (E); Vulnerabilidades: 62 (D); Code smells: 7.834 (A)
Coverage	Porcentaje de código cubierto por pruebas	46,0 %
Quality Gate	"Puerta de calidad": reglas que el código debe cumplir para avanzar	*Passed*
Leak period / New code	Mira solo lo que se agregó desde la versión anterior	Deuda nueva: 20 min; 3 issues nuevos

> **Respuesta modelo (lectura mía de las capturas).** Un mismo proyecto puede tener *Quality Gate: Passed* y calificación **A** en mantenibilidad, y a la vez **78,8 % de duplicación**, **calificación E en confiabilidad** y **46 % de cobertura**. Moraleja: **un indicador verde no cuenta toda la historia**; hay que mirar varias dimensiones (esto conecta con el "semáforo verde" del capítulo 6). Además, mirar solo el *código nuevo* permite que la deuda vieja no empeore, aunque no la paga.

El **gráfico "Accumulation of Technical Debt"** muestra 20 *releases*: lo que se agrega en cada versión es relativamente poco, pero la **deuda agregada total sube de manera sostenida** (aproximadamente de 100 a 320 ítems). Es la imagen del interés compuesto: **si no se paga, se acumula.**

6.5. 5.5 Cómo gestionar la deuda: hacerla visible

El título de la slide es la idea: **"Hacerla visible para poder gestionarla."** Son cuatro pasos:

Paso	Qué se hace	Con qué se hace (según la slide)
1. Detectar	Identificar señales de deterioro	Complejidad, duplicación, *code smells*
2. Registrar	Convertir la deuda en **trabajo explícito**	Ubicación, causa, impacto (en el *backlog*)
3. Priorizar	Evaluar riesgo, costo y valor afectado	Urgencia, esfuerzo, consecuencia (matriz riesgo × impacto)
4. Reducir	Refactorizar de forma **segura y sostenida**	Pruebas, capacidad del *sprint*, seguimiento

El **tablero de deuda técnica** muestra el recorrido *de riesgo a control*: **alto riesgo** (caos creciente) → **riesgo elevado** (impacto significativo) → **riesgo moderado** (deuda controlable) → **bajo control** (mejora continua).

Dos mensajes finales de la slide:

- **La deuda técnica que no se registra compite de manera invisible con cada nueva funcionalidad.** (Si nadie la anotó, nadie la prioriza, y cada nueva función termina más cara.)
- **Gestionarla no significa eliminarla por completo**, sino tomar decisiones conscientes y evitar que crezca sin control.

> **Ojo.** "Reducir" incluye **pruebas** como parte del paso: refactorizar sin red de seguridad (pruebas) es otra forma de crear problemas nuevos. Y los tiempos de reducción se negocian **dentro de la capacidad del sprint**: no es algo que se hace "cuando sobre tiempo".

6.6. 5.6 Para recordar

- **Deuda técnica** = deficiencias acumuladas en calidad interna que **encarecen** modificar, mantener y evolucionar el software. El **interés** = esfuerzo adicional de cada cambio.
- **Cuadrante:** excesiva/arriesgada vs. moderada × voluntaria vs. involuntaria. Importa *cómo se genera*, *cuánto riesgo* y *si hay plan de pago*.
- **Iceberg:** los costos visibles son la punta; los invisibles (confianza, reputación, desgaste, oportunidades) son la mayor parte.
- **Gestión:** Detectar → Registrar → Priorizar → Reducir. No se busca eliminarla del todo.

7. 6. Cumplir el proceso no garantiza la calidad (Aspecto 03)

7.1. 6.1 La tesis

El *Aspecto 03* se titula: **"Gestionar el proceso es parte de la solución. No es suficiente."**

- Un proceso ordenado **reduce errores**, pero **la calidad se confirma evaluando el producto.**
- **Cumplir el proceso $\neq$ garantizar la calidad.**
- Recordá el cuadro del capítulo 3: el proceso **influye** pero **no garantiza**; *el proceso crea las condiciones, el producto confirma la calidad.*

Dos escenarios de la presentación lo muestran con casos concretos. Conviene saber resolverlos, porque son el tipo de razonamiento que se pide en un parcial teórico-práctico.

7.2. 6.2 Escenario: el "semáforo verde"

El caso (resumen). Una empresa desarrolla una plataforma de **autogestión para una universidad**. Durante seis meses el director del proyecto informa al comité que **todos los indicadores están en verde**: el 95 % de las tareas del sprint se completan a tiempo, se hacen las reuniones de Scrum y las retrospectivas, cada requerimiento tiene responsable y estado en Jira, los entregables salen en fecha, el presupuesto se respeta, los riesgos están registrados y la documentación está al día. La dirección considera al proyecto "bien gestionado".

Al poner el sistema en producción, **durante la primera semana** aparecen los problemas:

- los estudiantes tardan demasiado en encontrar dónde hacer trámites o inscribirse;
- algunas páginas demoran **entre 8 y 12 segundos** en responder;
- desde celulares, varias funciones son difíciles de usar;
- el sistema **no es accesible** para usuarios con ciertas discapacidades;
- aparecen **errores cuando muchos estudiantes ingresan simultáneamente**;
- la Mesa de Ayuda recibe **cientos de consultas**;
- varios usuarios **prefieren seguir haciendo trámites presencialmente**;
- el reclamo más frecuente: *"la página es lenta y confusa"*.

En la reunión posterior, el director vuelve a mostrar el tablero (plazos, costos, sprints y entregables en verde) y pregunta: *"Si gestionamos correctamente todo el proyecto… ¿qué salió mal?"*

Datos adicionales: el equipo cumplió todos los eventos y artefactos del marco ágil; las **pruebas se realizaron al final de cada sprint de manera superficial**; **no se midieron métricas de calidad del producto ni de experiencia de usuario**; las decisiones de diseño se centraron en cumplir funcionalidades, **no en usabilidad ni performance**.

Respuesta modelo a las siete preguntas de la slide.

1. *¿El proyecto estuvo bien gestionado?* **En lo que mide la gestión, sí**: plazos, costos, sprints, entregables, riesgos y documentación estaban bajo control. Pero "bien gestionado" no equivale a "buen producto".
2. *¿El software obtenido es de calidad?* **No**: es lento, confuso, no accesible, difícil de usar en móviles, falla con carga y la gente prefiere no usarlo. Falla la perspectiva de **producto** (rendimiento, defectos), la de **personas usuarias** (usabilidad, accesibilidad) y la de **valor** (no logra reemplazar el trámite presencial).
3. *¿Qué estaba midiendo la organización?* **Cumplimiento del proceso y de la gestión** (plazos, costos, tareas completadas, ceremonias, documentación). Es decir, indicadores de **calidad del proceso**.
4. *¿Qué cosas importantes no estaba observando?* **Calidad del producto y experiencia de usuario**: tiempos de respuesta, comportamiento bajo carga, defectos, usabilidad, accesibilidad, uso real y satisfacción; tampoco se validó con usuarios reales.
5. *¿Es posible un proceso ordenado y un producto deficiente?* **Sí.** Seguir los eventos de Scrum no garantiza que se esté construyendo lo correcto ni que se construya bien. Las pruebas superficiales al final del sprint y el foco exclusivo en "cumplir funcionalidades" dejaron pasar los problemas.
6. *¿Qué debería incorporarse además de la gestión?* **Medición de calidad del producto** (rendimiento, errores, cobertura), **pruebas de performance/carga y de usabilidad/accesibilidad desde temprano**, criterios de calidad en la *Definition of Done*, *quality gates*, **validación con usuarios reales**, métricas de experiencia (satisfacción, éxito de tareas) y **decisiones basadas en evidencia**. Es decir: QA (prevenir en el proceso) *y* QC (verificar en el producto).
7. *Frase resumen:* "**Cumplir el proceso no garantiza la calidad**" (o "se gestionó el proyecto, no la calidad del producto").

7.3. 6.3 Escenario: "La empresa certificada"

El caso (resumen). Una empresa de software participa de una **licitación** para construir una plataforma de **gestión de turnos médicos**. En su presentación comercial destaca: **certificación de su sistema de gestión de calidad**, procesos de desarrollo documentados, procedimientos formales de gestión de cambios, auditorías internas periódicas, registros de incidencias y acciones correctivas, indicadores de cumplimiento de procesos y personal capacitado con roles definidos. El contratante lo considera una **garantía importante** y le adjudica el proyecto. El desarrollo se hace siguiendo los procedimientos, los documentos se generan, las revisiones se registran y **las auditorías internas no detectan incumplimientos significativos**.

Seis meses después, ya en producción, aparecen: **turnos duplicados**, lentitud en horarios de alta demanda, profesionales que **no pueden acceder desde dispositivos móviles**, pantallas confusas para usuarios mayores, errores al reprogramar turnos, **fallas intermitentes en la integración** con el sistema de notificaciones y numerosos reclamos en la mesa de ayuda.

En la reunión de seguimiento, el contratante pregunta: *"¿Cómo puede ocurrir esto si contratamos una empresa certificada y todos sus procesos fueron auditados?"* La empresa responde: *"Nuestros procesos fueron ejecutados conforme a los procedimientos establecidos y la auditoría no encontró incumplimientos."*

Respuesta modelo a las siete preguntas de la slide.

1. *¿La empresa incumplió necesariamente su sistema de gestión?* **No.** Según el relato, cumplió sus procedimientos y la auditoría no halló incumplimientos significativos. El problema no es un incumplimiento, sino que **cumplir no basta**.
2. *¿Es posible que el proceso se haya seguido bien y el producto tenga problemas?* **Sí**, es exactamente lo que ilustra el caso: el proceso influye pero no garantiza.
3. *¿Qué acredita realmente una certificación?* Que la organización tiene un **sistema de gestión de calidad definido, implementado y auditado** conforme a una norma. Acredita **capacidad de gestión de procesos**, no que cada producto cumpla todos sus requisitos.
4. *¿La certificación evalúa cada funcionalidad, rendimiento, usabilidad o confiabilidad del software entregado?* **No.** La auditoría verifica cumplimiento de procedimientos y registros (evidencia de proceso), no ejecuta ni mide el software.
5. *¿Qué otros mecanismos deberían haberse usado para evaluar la calidad del producto?* **Criterios de aceptación medibles en el contrato** (rendimiento, disponibilidad, compatibilidad móvil, accesibilidad, usabilidad), **pruebas** de sistema, carga e integración, **pruebas de usabilidad con usuarios reales**, pilotos o prototipos, auditoría técnica del producto, métricas de calidad del producto y aceptación formal (UAT).
6. *¿Qué error cometió la organización contratante al interpretar la certificación?* **Tomó una evidencia de calidad del proceso como si fuera garantía de calidad del producto**, y no definió ni verificó requisitos de calidad del producto.
7. *Frase resumen:* "**La certificación respalda al proceso; la calidad del producto se verifica en el producto.**"

Extra (no está en las slides). En la práctica, una certificación como ISO 9001 se otorga a un **sistema de gestión de calidad** de una organización, no a un producto concreto. Existen también certificaciones o evaluaciones **de producto** (con ensayos sobre el producto), pero son otra cosa. Y la slide de Principios lo dice con todas las letras: *implementar un SGC no significa necesariamente certificar una norma* (capítulo 8).

7.4. 6.4 Qué evidencia pedir: proceso vs. producto

Una forma práctica de no caer en la trampa es distinguir **qué tipo de evidencia** se está mirando:

Pregunta	Evidencia de **proceso** (QA)	Evidencia de **producto** (QC)
¿Se prueba el software?	Existe una estrategia de pruebas y *Definition of Done*	Resultados de las pruebas: cuántos casos pasan, defectos encontrados
¿Es rápido?	Hay un requisito de rendimiento definido	Mediciones de tiempo de respuesta bajo carga
¿Es usable?	El diseño incluye revisiones de usabilidad	Pruebas con usuarios reales: éxito de tareas, errores
¿Se controlan los cambios?	Procedimiento de gestión de cambios	Regresiones detectadas, defectos escapados
¿El equipo mejora?	Retrospectivas y acciones registradas	Tendencia de defectos, tiempo de ciclo, satisfacción

> **Idea clave.** Las dos columnas son necesarias. **Evidencia de proceso sin evidencia de producto** produce el "semáforo verde" que no ve los problemas. **Evidencia de producto sin proceso** detecta defectos pero no evita que vuelvan a aparecer.

7.5. 6.5 Para recordar

- **Cumplir el proceso $\neq$ garantizar la calidad.** El proceso crea las condiciones; el producto confirma la calidad.
- **Semáforo verde:** indicadores de gestión en verde pueden convivir con un producto deficiente si no se mide la calidad del producto ni la experiencia de usuario.
- **Empresa certificada:** una certificación acredita un **sistema de gestión**, no cada producto. Hay que definir y verificar la calidad **del producto** por separado.
- Para obtener calidad hacen falta **QA (prevenir en el proceso) y QC (verificar en el producto)**.

8. 7. Costos de la calidad y de la no calidad

> *"Invertir para prevenir, medir para decidir."* — lema de la presentación

8.1. 7.1 Qué son y para qué sirven

Los **costos de la calidad** son **los recursos invertidos para prevenir y evaluar defectos, más los costos provocados por fallas internas y externas.**

El **objetivo** de analizarlos no es "gastar menos" a ciegas, sino **identificar dónde se consume el dinero y decidir dónde conviene invertir.**

8.2. 7.2 Las cuatro categorías

Se ordenan siguiendo la vida de un defecto: se intenta **evitar** que aparezca, se lo **busca**, se lo **corrige antes de entregar** o, si se escapó, lo **sufre el usuario**.

Categoría	Qué es	Ejemplos de la cátedra	Ejemplos de software (aporte mío)
1. Prevención	Evitar que los defectos aparezcan	Capacitación, estándares, diseño de arquitectura, revisión temprana de requisitos	Guías de codificación, revisiones de diseño, formación del equipo en testing, prototipos
2. Evaluación	Comprobar que el producto cumple	Testing, auditorías, análisis, pruebas funcionales, automatización, análisis estático	Ejecutar pruebas planificadas, revisiones de código, análisis estático, auditorías de calidad
3. Fallas internas	Defectos detectados **antes** de la entrega	Retrabajo, correcciones, demoras	Corregir un bug hallado en pruebas de sistema, repetir pruebas tras un arreglo, descartar código
4. Fallas externas	Defectos detectados **por los usuarios** en producción	Incidentes, soporte, reputación, pérdida de clientes, hotfixes, indisponibilidad, compensaciones	Hotfix de urgencia, atención de reclamos, indemnizaciones, bajas de clientes

8.2.1. Regla práctica para clasificar un costo

Hacete estas preguntas en orden:

1. ¿Es un gasto **para evitar** que el defecto aparezca? → **Prevención.**
2. ¿Es un gasto **planificado para buscar** defectos antes de entregar? → **Evaluación.**
3. ¿Es el costo de **corregir un defecto encontrado antes** de entregar? → **Falla interna.**
4. ¿Es el costo de un defecto **que ya llegó al usuario**? → **Falla externa.**

> **Extra (no está en las slides).** Este esquema se conoce como modelo **PAF** (*Prevention – Appraisal – Failure*), asociado a Feigenbaum; Crosby lo expresa como "precio de la conformidad" y "precio de la no conformidad". Un detalle fino: las pruebas **planificadas** son evaluación, pero **repetir** pruebas porque hubo que corregir un defecto se suele contar como falla interna.

8.3. 7.3 Conformidad y no conformidad

Las cuatro categorías se agrupan en dos grandes bloques:

	Costos de conformidad	**Costos de no conformidad**
Incluye	Prevención + Evaluación	Fallas internas + Fallas externas
Naturaleza	**Inversión planificada** para evitar defectos o detectarlos antes de la entrega	**Costos reactivos** originados por productos o procesos que no cumplen lo esperado
Lema	"Invertir para prevenir y comprobar"	"Consecuencias de los defectos"
Ejemplos	Capacitación, revisión de requisitos, análisis de código y pruebas	Retrabajo, hotfixes, soporte, indisponibilidad y compensaciones

$$\text{Costo total de la calidad} = \text{Conformidad} + \text{No conformidad}$$

La frase que lo resume: **"La calidad se paga al prevenir y evaluar; la no calidad se paga al corregir y recuperar."** O, en el título de la slide: **la calidad tiene costo; la no calidad también.**

Ojo (confusiones clásicas).

- **"Costos de la calidad"** es el **total** (las cuatro categorías). **"Costo de la no calidad"** o **COPQ** (*Cost of Poor Quality*) es **solo** fallas internas + externas.
- Gastar en testing **no es un costo de no calidad**: es un costo de **conformidad** (evaluación). Lo que cuesta por no tener calidad es *corregir* lo que el testing encontró o lo que se escapó.
- La **pérdida de ventas por mala reputación** es un costo de la calidad (falla externa), aunque sea difícil de medir (es de los "invisibles" del capítulo 5).

Para el examen. En el parcial 2025 se preguntó cuáles son "costos de la calidad": *formación del personal en testing* (prevención), *inspecciones de código y revisiones de diseño* (evaluación) y *pérdida de ventas por mala reputación* (falla externa). Las tres pertenecen al modelo. Cuando una pregunta dice "marcar la/s correcta/s", **puede haber más de una**.

8.4. 7.4 Caso real: CrowdStrike (2024)

Para mostrar que la no calidad puede ser carísima, la presentación cita el caso **CrowdStrike**: una **actualización defectuosa** provocó una **interrupción global de sistemas Windows** y **pérdidas directas estimadas en US$ 5.400 millones entre empresas Fortune 500 afectadas** (fuente de la slide: CrowdStrike y Parametrix, 2024).

Idea clave. Es un ejemplo de **falla externa a escala masiva**: el defecto llegó a producción, y el costo no lo pagó solo el proveedor, sino los clientes. Un solo cambio sin evaluación suficiente puede costar muchísimo más que haber invertido en prevenir y evaluar.

Extra. El incidente ocurrió el 19 de julio de 2024.

8.5. 7.5 Cómo medir el costo de la no calidad: COPQ

COPQ (*Cost of Poor Quality*) es el **costo total ocasionado por productos, procesos o servicios que no cumplen con la calidad esperada** (en español: costo de la mala calidad o costo de la no calidad).

$$\text{COPQ} = \text{Fallas internas} + \text{Fallas externas}$$

$$\text{COPQ sobre ingresos (\%)} = \frac{\text{COPQ}}{\text{Ingresos totales}} \times 100$$

$$\text{Fallas internas sobre COPQ (\%)} = \frac{\text{Fallas internas}}{\text{COPQ}} \times 100$$

$$\text{Fallas externas sobre COPQ (\%)} = \frac{\text{Fallas externas}}{\text{COPQ}} \times 100$$

$$\text{Costo por cliente} = \frac{\text{COPQ}}{\text{Clientes activos}}$$

8.5.1. Ejemplo de la slide: plataforma SaaS (período anual)

Datos: ingresos US$ 500.000; fallas internas US$ 12.000; fallas externas US$ 28.000; clientes activos 10.000.

Paso	Cálculo	Resultado	Cómo se lee
COPQ	12.000 + 28.000	**US$ 40.000**	Lo que se pierde por no calidad en el año
COPQ sobre ingresos	(40.000 ÷ 500.000) × 100	**8 %**	De cada US$ 100 que ingresan, US$ 8 se pierden por fallas
Origen: internas	(12.000 ÷ 40.000) × 100	**30 %**	Se detectó antes de entregar
Origen: externas	(28.000 ÷ 40.000) × 100	**70 %**	**El 70 % del COPQ ocurre en producción**
Costo por cliente	40.000 ÷ 10.000	**US$ 4**	Por cliente y por año

Lectura clave de la slide: el COPQ muestra **cuánto pierde la organización por fallas y dónde se concentra esa pérdida.** Acá, la mayor parte (70 %) se concentra en fallas externas: el problema es que los defectos se están escapando hacia los usuarios.

8.6. 7.6 Indicadores económicos de la calidad

La cátedra presenta cinco indicadores. Los números del ejemplo posterior son míos, para que veas cómo se interpretan.

Indicador	Fórmula	Qué indica	Conviene
1. COPQ sobre el proyecto	(Fallas internas + Fallas externas) ÷ Costo total del proyecto × 100	Qué proporción del presupuesto se pierde por no calidad	Bajo
2. Costo promedio por defecto	Costo total asociado a fallas ÷ Cantidad total de defectos	Impacto económico medio de cada defecto	Bajo

Indicador	Fórmula	Qué indica	Conviene
3. Tasa de defectos escapados (*Escape rate*)	Defectos en producción ÷ Total de defectos detectados × 100	Qué porcentaje de defectos llegó hasta los usuarios	Bajo
4. Eficiencia de remoción de defectos (DRE)	Defectos eliminados antes de producción ÷ Total de defectos detectados × 100	La capacidad de detectar y corregir **antes de liberar**	Alto
5. Tasa de retrabajo	Horas de retrabajo ÷ Horas totales del proyecto × 100	Cuánto esfuerzo se consume **corrigiendo trabajo ya realizado**	Bajo

> **Idea clave (cierre de la slide).** *"Un indicador aislado describe una señal. Su evolución en el tiempo permite gestionar la calidad."* Un dato suelto no dice si estás mejorando; la tendencia sí.

> **Extra (relación entre dos indicadores).** Como ambos usan el mismo denominador (total de defectos detectados), **DRE + Escape rate = 100 %**. Si el DRE es 85 %, el escape rate es 15 %. Son dos formas de mirar lo mismo.

Ejemplo propio (verificado): un proyecto detectó **200 defectos** en total y **30** se encontraron en producción.

- Escape rate = 30 ÷ 200 × 100 = **15 %**.
- DRE = (200 — 30) ÷ 200 × 100 = 170 ÷ 200 × 100 = **85 %**.
- Si el costo total asociado a fallas fue US$ 75.000: costo por defecto = 75.000 ÷ 200 = **US$ 375**.
- Si se dedicaron 480 horas de retrabajo sobre 3.200 horas totales: retrabajo = 480 ÷ 3.200 × 100 = **15 %**.

8.7. 7.7 Caso práctico: Proyecto Ágora

Enunciado. Ágora es una plataforma SaaS de gestión académica. Presupuesto total del proyecto: **US$ 200.000**. Costos asociados a la calidad durante una versión anual:

Categoría	Conceptos incluidos	Costo
Prevención	Capacitación, diseño de arquitectura y revisión temprana de requisitos	US$ 8.000
Evaluación	Pruebas funcionales, automatización y análisis estático	US$ 12.000
Fallas internas	Retrabajo y correcciones detectadas antes de la liberación	US$ 18.000

Categoría	Conceptos incluidos	Costo
Fallas externas	Incidentes en producción, soporte y pérdida de clientes	US$ 32.000

Resolución (tablero de la slide, con los pasos explícitos):

Concepto	Cálculo	Resultado
Costo total de calidad	8.000 + 12.000 + 18.000 + 32.000	**US$ 70.000**
… como % del presupuesto	70.000 ÷ 200.000	**35 %**
Costos de conformidad	8.000 + 12.000	**US$ 20.000**
… como % del costo de calidad	20.000 ÷ 70.000	**28,6 %**
Costos de no conformidad (COPQ)	18.000 + 32.000	**US$ 50.000**
… como % del costo de calidad	50.000 ÷ 70.000	**71,4 %**
Distribución	Prevención 8.000 ÷ 70.000	11,4 %
	Evaluación 12.000 ÷ 70.000	17,1 %
	Fallas internas 18.000 ÷ 70.000	25,7 %
	Fallas externas 32.000 ÷ 70.000	45,7 %
Señal crítica	Fallas externas ÷ COPQ = 32.000 ÷ 50.000	**64 %**

La "señal crítica" de la slide: las fallas externas son el **64 % del COPQ**: US$ 32.000 de US$ 50.000 se originan **cuando el defecto ya llegó a producción**. Y concluye: *"La mayor oportunidad no está en evaluar más, sino en prevenir y detectar antes."*

> **Respuesta modelo (por qué "prevenir y detectar antes" y no "evaluar más").**
>
> - Ya se invierte en evaluación (US$ 12.000, 17,1 %), y aun así **llegan a producción defectos que cuestan US$ 32.000**. Sumar pruebas *al final* encuentra los defectos tarde y no impide que se generen.
> - La prevención recibe solo **11,4 %** del costo de calidad. Mejores requisitos, arquitectura y capacitación evitan defectos y reducen **a la vez** fallas internas y externas.
> - Detectar antes (en etapas tempranas) hace que el mismo defecto se corrija **cuando todavía es barato**, en lugar de pagarlo como falla externa.
> - Dato mío: no conformidad ÷ conformidad = 50.000 ÷ 20.000 = **2,5**: por cada dólar invertido en prevenir y evaluar, se gastan **US$ 2,5** en corregir y recuperar.

> **Ojo (dos porcentajes que se confunden).** La slide muestra que el **costo total de calidad es 35 % del presupuesto** (70.000 ÷ 200.000). Pero el **indicador "COPQ sobre el proyecto"** de la slide anterior usa solo las fallas: aplicándolo da (18.000 + 32.000) ÷ 200.000 × 100 = **25 %** (cálculo mío). No son lo mismo: uno incluye prevención y evaluación; el otro no. Y el "COPQ sobre ingresos" del ejemplo SaaS divide por **ingresos**, no por presupuesto. **Siempre fijate cuál es el denominador.**

8.8. 7.8 Para recordar

- **Cuatro categorías:** prevención, evaluación (**conformidad**) y fallas internas, fallas externas (**no conformidad = COPQ**).
- **Costo total de la calidad = conformidad + no conformidad.**
- **COPQ = FI + FE.** Fórmulas: COPQ/ingresos, FI/COPQ, FE/COPQ, costo por cliente.
- **Cinco indicadores:** COPQ % del proyecto, costo por defecto, escape rate, DRE, tasa de retrabajo. Un indicador aislado es una señal; la **evolución** permite gestionar.
- **Mensaje central:** cuando las fallas externas dominan, hay que **prevenir y detectar antes**.

9. 8. Calidad Total y los ocho principios

9.1. 8.1 ¿Alcanza con que el software funcione?

La presentación abre con una imagen: una pantalla muestra **"Pago aprobado"**… pero alrededor hay una caja abierta que no llegó bien, un paquete cuya entrega ya está fuera de fecha, una persona frustrada hablando con soporte y un equipo desbordado. El software "funciona", pero **la experiencia completa falla**. La calidad no se agota en que una función ande: involucra a clientes, equipos y a toda la organización. De ahí la idea de **calidad total**.

9.2. 8.2 Definición de Calidad Total

> La calidad total se define como la **excelencia** en los productos o servicios que satisface las **expectativas del cliente**, tanto **interno** como **externo**, lograda con el **menor costo posible** y en armonía con el **entorno social**, dentro de un **proceso continuo**. Esto se debe, entre otras causas, a que las expectativas de los clientes son **cambiantes** y presentan niveles de exigencia **cada vez mayores**, teniendo como **objetivo final la supervivencia de la organización**.

La slide siguiente la desglosa en cinco ideas:

N.º	Idea	Qué significa
1	**Excelencia** en productos y servicios	No solo "cumplir": buscar lo mejor posible
2	**Satisfacción** del cliente interno y externo	Quien recibe mi trabajo es mi cliente, dentro o fuera de la organización
3	**Eficiencia** con el menor costo posible	Calidad **sin desperdicio**; conecta con el capítulo 7
4	**Compromiso** con el entorno social	La organización actúa en armonía con la sociedad
5	**Mejora continua** ante expectativas cada vez más exigentes	Lo que hoy satisface, mañana puede no alcanzar

Objetivo final: la **supervivencia y sostenibilidad** de la organización.

Extra (para entenderlo mejor).

- **Cliente externo:** quien recibe el producto o servicio fuera de la organización (la persona usuaria, el cliente que paga). **Cliente interno:** quien recibe el trabajo de otro dentro de la organización (testing recibe el trabajo de desarrollo; soporte "recibe" el producto con sus defectos).
- Se dice **"total"** porque abarca a **toda** la organización (no solo al área de calidad), **todas** las etapas y **todas** las personas.
- Que el objetivo final sea la **supervivencia** explica el resto: si las expectativas suben y la organización no mejora, eventualmente deja de ser competitiva.

9.3. 8.3 Los ocho principios, de un vistazo

Roadmap de la clase (el subtítulo de cada principio) y pregunta orientadora (de la Guía del TP de Auditoría Forense):

N.º	Principio	Idea en una línea (roadmap)	Pregunta orientadora (Guía del TP)
1	**Orientación al cliente**	Comprender y crear valor	Comprender necesidades y evaluar el valor percibido
2	**Liderazgo** (y compromiso)	Dar dirección y propósito	Marcar el rumbo, dar el ejemplo y brindar condiciones
3	**Participación del personal**	Comprometer capacidades	Dar voz, autonomía y responsabilidad a las personas
4	**Enfoque basado en procesos**	Gestionar cómo se crea valor	Gestionar el flujo completo y sus interacciones
5	**Enfoque de sistema / Implementación de un SGC**	Conectar procesos y objetivos	Organizar la calidad como una capacidad estable
6	**Mejora continua**	Aprender y evolucionar	Aprender, experimentar y sostener mejores resultados
7	**Decisiones basadas en evidencia**	Decidir con datos y criterio	Combinar datos confiables con criterio experto
8	**Relación con proveedores**	Construir calidad en conjunto	Construir acuerdos, confianza y mejora conjunta

La frase que cierra el roadmap: **"Del valor esperado por el cliente a un sistema que aprende y mejora."**

> **Ojo (el nombre del principio 5).** En el roadmap figura como **"Enfoque de sistema"**, pero las slides del principio y la Guía del TP lo llaman **"Implementación de un Sistema de Gestión de la Calidad (SGC)"**. Es el mismo principio; el contenido que desarrolla la cátedra es el de implementar un SGC. En el examen, nombralo como **"Enfoque de sistema / Implementación de un SGC"**.

> **Extra (de dónde salen los ocho).** Son los ocho principios de gestión de la calidad de la familia ISO 9000 hasta la versión 2005 (clientes, liderazgo, participación del personal, procesos, enfoque de sistema para la gestión, mejora continua, decisiones basadas en hechos, relaciones con proveedores). La versión ISO 9000:2015 los reorganizó en **siete** (el enfoque de sistema dejó de ser un principio aparte). La cátedra trabaja con ocho, con nombres actualizados en algunos (por ejemplo "decisiones basadas en evidencia").

Cómo memorizarlos (aporte mío). Un recorrido lógico: primero *para quién* (1 cliente), luego *quién guía* (2 liderazgo), *quién lo hace* (3 personas), *cómo se organiza el trabajo* (4 procesos) y *el sistema que lo sostiene* (5 SGC), después *cómo aprende* (6 mejora, 7 evidencia) y, por último, *quién más participa desde afuera* (8 proveedores).

9.4. 8.4 Principio 1 — Orientación al cliente

"La calidad comienza por comprender qué necesita lograr el cliente." — No diseñamos funciones: diseñamos experiencias que deben generar valor.

- **Qué significa:** comprender **necesidades, expectativas y contexto de uso** para diseñar una solución que ayude al cliente a alcanzar un resultado.
- **Por qué importa:** una función **técnicamente correcta** puede fracasar si resulta **confusa, lenta, innecesaria o no resuelve el problema real**.
- **Recorrido de experiencia del usuario:** *descubre → explora → usa → logra → recomienda.* Cada etapa aporta una pieza distinta del valor, y las métricas se **complementan** para ver el panorama completo.

¿Cómo medirlo? Tres preguntas y sus métricas:

Pregunta	Métricas de la slide	Qué mide
1. ¿Logra?	Éxito de tareas, errores, tiempo de realización	Si la persona **consigue su objetivo**
2. ¿Cómo lo vive?	CSAT, CES, reclamos y tickets	La experiencia y la satisfacción
3. ¿Vuelve a elegir?	Conversión, abandono, retención	El comportamiento a lo largo del tiempo

> **Extra. CSAT** (*Customer Satisfaction Score*): cuán satisfecha quedó la persona (suele medirse con una encuesta corta). **CES** (*Customer Effort Score*): cuánto esfuerzo le costó lograr lo que quería.

Frase clave de la slide: *"No alcanza con preguntar si está satisfecho: hay que **observar si logra su objetivo**."*

Caso (hipotético): Spotify actualiza su experiencia. Se lanza una **nueva pantalla de inicio personalizada**; el **tiempo de escucha aumenta 12 %**; pero **muchos usuarios reclaman que ya no encuentran fácilmente sus playlists**; el equipo **mantiene el cambio y promete evaluarlo dentro de tres meses**. Pregunta: ¿se cumple el principio 1? ¿Qué evidencias sostienen la respuesta?

> **Respuesta modelo. No se cumple plenamente.**
>
> - *Evidencia a favor:* hubo una mejora medida (+12 % de tiempo de escucha).
> - *Evidencias en contra:* (1) la métrica elegida mide **uso**, no si la persona **logra lo que necesita** (encontrar sus playlists); (2) los **reclamos** son evidencia directa de problemas en las dimensiones "¿logra?" y "¿cómo lo vive?"; (3) **posponer tres meses** la evaluación implica no actuar sobre una señal temprana.
> - *Qué haría una organización orientada al cliente:* medir éxito y tiempo para encontrar una playlist, analizar reclamos y tickets, probar con usuarios, considerar una prueba A/B o ajustes parciales y definir criterios para decidir si se mantiene el cambio.
> - *Matiz:* el +12 % no prueba que el cambio sea malo; prueba que **la evidencia disponible no alcanza para afirmar que sea bueno**.

> **Para el examen.** El parcial 2025 preguntó qué implica el "enfoque al cliente" en TQM. La respuesta correcta: **entender y satisfacer tanto las necesidades actuales como las futuras de los clientes internos y externos.** Descartaba "priorizar *únicamente* lo que los clientes piden explícitamente" (ignora lo implícito) y "implementar un sistema de tickets" (es una herramienta, no el principio).

9.5. 8.5 Principio 2 — Liderazgo y compromiso con la calidad

"La calidad empieza con el ejemplo."

Qué significa liderar la calidad: la dirección debe convertir la calidad en una **prioridad compartida y visible en las decisiones cotidianas.** Cinco acciones:

Acción	Qué implica
Marcar el rumbo	Definir propósito, objetivos y prioridades claras
Dar el ejemplo	Actuar con coherencia entre lo que se dice y lo que se hace
Involucrar	Escuchar, motivar y hacer partícipe a todo el equipo

Acción	Qué implica
Dar recursos	Brindar tiempo, formación y herramientas para trabajar con calidad
Mejorar	Revisar resultados, aprender de los errores y sostener los cambios

Frase clave: *"La calidad no se exige solamente: se **lidera**, se **facilita** y se **demuestra**."*

Cinco señales de que el principio 2 NO se cumple:

1. **Discurso sin ejemplo:** se habla de calidad, pero las decisiones priorizan siempre el plazo.
2. **Falta de rumbo:** no hay objetivos de calidad claros ni responsabilidades definidas.
3. **Sin recursos:** no se asignan tiempo, formación ni herramientas para mejorar.
4. **Equipo sin voz:** las alertas y propuestas del personal no son escuchadas.
5. **Cultura de culpa:** se buscan responsables en lugar de causas y aprendizajes.

Escenario de la slide: la dirección exige entregar rápido, pero no participa en las decisiones de calidad, posterga las mejoras y responsabiliza al equipo cuando aparecen errores. Pregunta final: *¿la calidad es una prioridad real o solamente un mensaje?* **Tip:** observá qué hace la dirección **cuando la calidad compite con la urgencia**.

> **Respuesta modelo.** Se incumplen las cinco señales casi a la vez: discurso sin ejemplo (exige rapidez pero no participa), falta de recursos (posterga las mejoras), y cultura de culpa (responsabiliza al equipo). La calidad es hoy "un mensaje": en el momento de elegir entre calidad y urgencia, gana la urgencia.

9.6. 8.6 Principio 3 — Participación del personal

"La calidad se construye entre todos."

Qué significa: todas las personas, **en todos los niveles**, deben sentirse **valoradas, involucradas y responsables** de aportar a la calidad. Cinco componentes:

Componente	Significado
Tener voz	Poder expresar ideas, alertas y propuestas
Comprender el aporte	Reconocer cómo el propio trabajo impacta en el resultado
Asumir responsabilidad	Comprometerse con la calidad, no solo con cumplir una tarea
Compartir conocimiento	Aprender entre pares y colaborar más allá de cada rol
Tener autonomía	Contar con confianza y capacidad para actuar y mejorar

Frase clave: *"Participar no es solamente estar: es poder **aportar, decidir y transformar**."*

Caso real (vigente 2026, fuente: Atlassian): ShipIt, de una idea a una solución en 24 horas. Atlassian ofrece a su personal un espacio para impulsar mejoras, resolver problemas o crear nuevas funciones. Tres pasos: **elegir** (cada persona trabaja en una idea que considera valiosa), **colaborar** (se forman equipos con conocimientos de distintas áreas) y **crear** (en 24 horas convierten la propuesta en un prototipo y la presentan). **Resultado real:** un portal más simple para crear incidencias dio origen a **Jira Service Desk**, hoy **Jira Service Management**.

> **Respuesta modelo (por qué cumple el principio 3).** Porque la empresa brinda **voz** (cualquiera puede proponer), **autonomía** (cada persona elige su idea), **tiempo** (24 horas dedicadas) y **colaboración** (equipos multidisciplinarios) para que las ideas del personal **se transformen en mejoras**: es poder *aportar, decidir y transformar*, no solo asistir.

9.7. 8.7 Principio 4 — Enfoque basado en procesos

"La calidad no vive en una tarea: fluye a través de todo el proceso."

Qué significa: gestionar las actividades relacionadas **como un sistema** permite transformar **entradas** en **resultados** de manera más eficaz, previsible y mejorable.

Entradas	→	Actividades relacionadas	→	Salidas
Necesidades, información y recursos		Tareas coordinadas que agregan valor		Productos, servicios y resultados

Con **mejora** como retroalimentación (*qué se aprende y qué se ajusta*) y cuatro elementos para gestionarlo: **responsables** (quién decide, ejecuta y acompaña), **recursos** (con qué personas, tiempo y herramientas), **controles** (cómo se mide y verifica el desempeño) e **interacciones** (cómo cada etapa afecta a las demás).

Frase clave: *"No alcanza con optimizar cada tarea por separado: hay que **mejorar el flujo completo**."*

Caso real de transformación digital en turismo. *Antes:* las reservas se registraban en Excel y la misma reserva se cargaba **manualmente en otro sistema** para ventas → consecuencias: **duplicación de tareas, errores, demoras y pérdida de competitividad**. *Transformación:* no se digitalizó una tarea aislada: se **rediseñó el flujo completo**. *Después:* **reserva → información única → venta**, con datos que fluyen sin recargarse manualmente → **menos errores, menos demoras, mayor capacidad de respuesta.**

> **Idea clave del caso.** *Transformar digitalmente no es cambiar una herramienta: es repensar el proceso de punta a punta.* El principio 4 está en **entender reservas y ventas como partes relacionadas de un mismo proceso orientado al cliente**.

Ocho tips para identificar y administrar procesos (slide):

1. **Definí el propósito:** ¿qué necesidad atiende y qué valor debe generar?
2. **Marcá los límites:** ¿qué evento lo inicia y qué resultado indica que terminó?
3. **Identificá entradas y salidas:** ¿qué recibe, de quién y qué entrega al cliente?
4. **Seguí el flujo real:** observá cómo se trabaja, no solo cómo está documentado.
5. **Asigná responsabilidad:** un responsable del **proceso completo**, no solo de cada tarea.
6. **Analizá interacciones:** pases entre áreas, sistemas, personas y otros procesos.
7. **Buscá desperdicios:** duplicaciones, esperas, retrabajo, errores y controles innecesarios.
8. **Medí el desempeño:** tiempo, calidad, costo, capacidad y satisfacción del cliente.

Tip ingenieril: si no podés señalar con claridad dónde comienza y dónde termina, todavía no delimitaste el proceso. Y: **antes de automatizar, entendé, modelá y mejorá el proceso.**

9.8. 8.8 Principio 5 — Enfoque de sistema / Implementación de un SGC

"La calidad deja de depender de esfuerzos aislados y se convierte en una forma organizada de gestionar."

Qué significa implementar un SGC (Sistema de Gestión de la Calidad): diseñar una **forma sistemática de dirigir y controlar la organización** para **asegurar y mejorar continuamente la calidad.** Se apoya en el ciclo **PDCA** (*planificar – hacer – verificar – mejorar*) y en estos elementos:

Elemento	Para qué sirve
Política y objetivos	Definen el rumbo y lo que se busca lograr
Roles y responsabilidades	Aseguran claridad y compromiso en toda la organización
Indicadores	Permiten medir el desempeño y tomar decisiones
Procesos	Transforman entradas en resultados con valor
Información documentada	Establece lo acordado y conserva el conocimiento
Auditorías y verificación	Evalúan el cumplimiento y la eficacia del SGC
Mejora	Impulsa acciones para eliminar causas y aumentar el valor para clientes y partes interesadas

¿Qué significa? Cuatro verbos: **alinear** (conectar la política y los objetivos de calidad con la estrategia), **organizar** (definir procesos, responsabilidades, recursos y documentación), **controlar** (medir resultados, gestionar riesgos y verificar el cumplimiento) y **aprender** (analizar desvíos, corregir causas y convertir la experiencia en mejora).

¿Por qué es importante? Cinco beneficios: **coherencia** (las personas trabajan con criterios compartidos), **previsibilidad** (los resultados dependen menos de la improvisación), **evidencia** (las decisiones se apoyan en datos y registros confiables), **confianza** (respuestas más consistentes

para clientes y partes interesadas) y **sostenibilidad** (las mejoras se mantienen aunque cambien las personas).

> **Importante (nota de la slide, y trampa de examen). Implementar un SGC no significa necesariamente certificar una norma. La certificación puede ser una decisión posterior.** Un SGC convierte la calidad en **una capacidad de la organización**, no en el esfuerzo individual de algunas personas. (Conecta con el caso de la "empresa certificada" del capítulo 6.)

9.9. 8.9 Principio 6 — Mejora continua

"Mejorar no es llegar a una meta: es aprender a avanzar siempre."

Qué es: la **búsqueda sistemática y permanente de mejores formas de trabajar**, basada en **datos, aprendizaje y pequeños o grandes cambios.** Aporta cuatro cosas: **adaptación** (responder a nuevas necesidades y contextos), **calidad** (reducir errores, variación y retrabajo), **eficiencia** (aprovechar mejor tiempo, recursos y conocimiento) y **aprendizaje** (convertir resultados y fallas en decisiones mejores).

9.9.1. Cuatro modelos de mejora continua

Modelo	Etapas	Para qué se usa
PDCA	Planificar → Hacer → Verificar → Actuar	Probar cambios, evaluar resultados y **estandarizar lo que funciona**
Kaizen	Pequeñas mejoras + participación diaria	Construir una cultura donde **todas las personas** mejoran el trabajo cotidiano
Six Sigma – DMAIC	Definir → Medir → Analizar → Mejorar → Controlar	**Problemas complejos** con variación, defectos y evidencia cuantitativa
Retrospectivas ágiles	Observar → Reflexionar → Acordar → Experimentar	Que los equipos **revisen periódicamente cómo trabajan** y acuerden mejoras

¿Cuál elegir? Depende del **problema**, la **evidencia disponible**, el **alcance** y la **velocidad de aprendizaje** necesaria. Mensaje final: *mejorar continuamente es **hacer del aprendizaje una práctica de gestión**.*

Extra (para entender cada modelo).

- **PDCA** (ciclo de Deming): *Planificar* (definir objetivo y cómo lograrlo), *Hacer* (ejecutar, idealmente a pequeña escala), *Verificar* (medir y comparar con lo esperado), *Actuar/Mejorar* (estandarizar si funcionó o ajustar y volver a empezar). Nota: la slide del principio 5 llama a la última fase **"Mejorar"** y la del principio 6 **"Actuar"**: es la misma etapa. El ciclo se originó con Shewhart y lo popularizó Deming. En el parcial 2025: *"un pilar de la Calidad Total según Deming es la mejora continua basada en el ciclo PDCA".*
- **Kaizen:** palabra japonesa que significa "cambio para mejor"; énfasis en mejoras pequeñas, constantes y de todos.
- **Six Sigma:** metodología para reducir la variación y los defectos de un proceso con enfoque estadístico.
- **Retrospectivas:** reuniones periódicas del equipo (típicas de Scrum) para revisar y mejorar la forma de trabajar. La carpeta de la materia incluye la técnica *Starfish.*

9.9.2. Six Sigma / DMAIC aplicado al software: "¿por qué llegan fallas a producción?"

La slide recorre las cinco fases con una pregunta cada una:

Fase	Pregunta	Qué se hace
D — Definir	¿Qué le duele al usuario?	Delimitar el incidente y su impacto
M — Medir	¿Qué está ocurriendo realmente?	Construir una **línea base con datos**
A — Analizar	¿Qué causa el problema?	Contrastar hipótesis y encontrar la **causa raíz**
I — Mejorar	¿Qué cambio puede resolverlo?	Probar una solución **en pequeña escala**
C — Controlar	¿Cómo evitamos volver atrás?	Monitorear, estandarizar y sostener

Las **evidencias de software** que se usan: defectos escapados, retrabajo, *lead time*, incidentes y satisfacción. Mensaje: *"El valor no está en contar bugs: está en descubrir y mejorar el sistema que los genera."* Se usa **cuando el problema se repite, puede medirse y sus causas no son evidentes.**

Ejemplo DMAIC (escenario hipotético): fallas en el checkout.

Fase	Qué pasó en el ejemplo
Definir	El checkout registra **8 incidentes cada 100 despliegues** y afecta ventas y confianza
Medir	Se analizan **12 semanas**: incidentes, módulos modificados, pruebas y momento de detección

Fase	Qué pasó en el ejemplo
Analizar	El **70 %** de las fallas aparece cuando los cambios en pagos **llegan tarde a las pruebas integrales**
Mejorar	Se incorporan **contratos de API**, **pruebas automáticas del flujo crítico** y una **regla de despliegue**
Controlar	Un **tablero semanal** monitorea incidentes, cobertura del checkout y cumplimiento de la regla

La **regla de despliegue** bloquea el despliegue si falla alguna prueba del flujo crítico o del contrato de API, o si la cobertura del checkout es menor al 80 %. **Resultado:** de **8 a 2 incidentes cada 100 despliegues** (**reducción del 75 %**, porque $(8-2)/8 = 0{,}75$).

> **Idea clave.** Lo que aporta Six Sigma: **no se corrigió solamente el último bug**; se identificó y mejoró el **patrón del proceso** que generaba las fallas. Se pasa de una percepción general a una causa comprobada, una solución verificable y un resultado sostenido.

9.10. 8.10 Principio 7 — Toma de decisiones basada en evidencia

"Los datos no reemplazan el criterio: lo hacen más sólido."

Definición de la slide: decidir con evidencia es usar **información pertinente, confiable y analizada en contexto, junto con el conocimiento experto.** Es pasar **de la opinión a una decisión fundamentada.**

¿Por qué es importante? Reduce sesgos, **anticipa riesgos**, **justifica prioridades** y **genera aprendizaje.**

Cinco pasos y sus herramientas:

Paso	Pregunta o foco	Herramientas (slide)
1. Preguntar	¿Qué decisión debemos tomar?	Preguntas SMART, *problem statement*
2. Reunir	Datos, métricas y voz del usuario	Entrevistas, encuestas, logs, KPI
3. Validar	Calidad, fuente y contexto	Triangulación, *checklist* de calidad
4. Interpretar	Patrones, causas e incertidumbre	Pareto, 5 porqués, Ishikawa
5. Decidir y verificar	Actuar y medir el resultado	Matriz multicriterio, prueba A/B, KPI

Extra (glosario rápido de las herramientas).

- **SMART:** pregunta/objetivo *Específico, Medible, Alcanzable, Relevante y con Tiempo definido.*
- **Triangulación:** contrastar un dato con al menos otra fuente o método independiente.
- **Pareto:** pocas causas explican la mayoría de los efectos (idea del "80/20").
- **5 porqués:** preguntar "¿por qué?" repetidamente hasta llegar a la causa raíz.
- **Ishikawa** (espina de pescado): organiza las causas posibles de un problema por categorías.
- **Matriz multicriterio:** comparar opciones puntuando cada una según varios criterios.
- **Prueba A/B:** comparar dos variantes con usuarios reales para ver cuál rinde mejor.

Fórmula de la slide: *evidencia + criterio experto → mejores decisiones.* Y una advertencia: **no todo lo medible es importante, ni toda evidencia es solo numérica.**

9.10.1. Diagrama de Ishikawa

Es una **herramienta visual que organiza las causas posibles de un problema para analizarlo de manera sistemática.** Las categorías que muestra la slide son **Personas, Procesos, Tecnología, Datos, Entorno y Medición.** Cómo se engancha con el principio 7:

1. **Observar:** definir el problema **con datos**.
2. **Formular hipótesis:** ordenar las causas posibles (en el diagrama, puntos **rojos** = causas posible s/hipótesis).
3. **Validar:** buscar evidencia antes de decidir (puntos **verdes** = causas validadas).

Idea clave. *El diagrama **propone hipótesis**; la evidencia **confirma o descarta** causas.* Y **no busca culpables: ayuda a comprender el sistema que produce el problema.**

Ejemplo: fallas en el checkout (del síntoma a una causa raíz comprobada). Problema medido: **8 incidentes cada 100 despliegues**.

Categoría	Causa posible (hipótesis)
Personas	Comunicación tardía de cambios en pagos
Proceso	Pruebas integrales demasiado tarde
Tecnología	Contratos de API incompatibles
Datos	Datos de prueba desactualizados
Entorno	*Staging* diferente de producción
Medición	Sin trazabilidad de punta a punta

Evidencia reunida: *logs* (70 % de los incidentes ocurre en pagos), *Git* (los cambios llegan después de las pruebas integrales), *tests* (fallan los contratos entre checkout y la API de pagos), comparación (*staging* no reproduce todas las versiones). **Causa raíz validada:** cambios tardíos en la API de pagos + ausencia de pruebas de contrato. **Decisión basada en evidencia:** incorporar *contract testing*, integrar pagos antes y bloquear el despliegue si falla el flujo crítico.

> **Ojo (el mismo escenario, dos lentes).** Este ejemplo y el de DMAIC (principio 6) cuentan **el mismo problema hipotético del checkout**. En el principio 6 se mira el **método de mejora** (DMAIC); en el 7, **cómo se valida la causa** (Ishikawa + evidencia). Además, dos cifras del 70 % no son lo mismo: en DMAIC, "el 70 % de las fallas aparece cuando los cambios en pagos llegan tarde a las pruebas integrales"; en Ishikawa, "el 70 % de los incidentes ocurre en pagos" (*logs*). Son hallazgos distintos, y sostienen juntos la misma causa raíz.

9.11. 8.11 Principio 8 — Relación con los proveedores

"La calidad del software también se construye fuera de la organización." — Un producto es tan confiable como el ecosistema que lo sostiene.

Qué significa: gestionar a los proveedores **como aliados del sistema de calidad**, con **requisitos claros, confianza, evidencia y mejora conjunta. No se compra solo tecnología: también se incorporan riesgos y capacidades.**

¿Quiénes son los proveedores en software? Nube (infraestructura), **APIs externas**, **pasarelas de pago**, servicios de identidad, **bibliotecas de código abierto**, servicios de ciberseguridad, soporte técnico.

¿Para qué? Continuidad del servicio, **seguridad y cumplimiento**, **menos fallas en integraciones** y **mejor experiencia del usuario.**

Gestión en cinco pasos (slide general): 1. Seleccionar (evaluar capacidad, seguridad y continuidad) → **2. Acordar** (definir SLA, criterios de calidad y responsabilidades) → **3. Integrar** (validar compatibilidad, datos y contingencias) → **4. Medir** (monitorear disponibilidad, defectos y respuesta) → **5. Mejorar juntos** (compartir hallazgos y prevenir recurrencias).

Ejemplo: tienda online + pasarela de pagos. La Organización A es responsable del sitio y de la experiencia de compra; la Organización B provee el procesamiento de pagos. **Problema:** algunos pagos fallan y los clientes abandonan la compra. La relación se gestiona así: **1. Acordar** (disponibilidad, seguridad y tiempos de respuesta) → **2. Integrar** (API documentada y responsabilidades claras) → **3. Probar** (casos reales en un entorno de prueba) → **4. Medir** (tasa de pagos exitosos e incidentes) → **5. Mejorar juntos** (analizar causas y prevenir recurrencias).

> **Idea clave. La falla del proveedor también se convierte en experiencia del cliente.** El cliente no distingue quién tuvo la culpa: ve que *tu* producto falla. Por eso **el proveedor deja de ser un vendedor y se convierte en un aliado de la calidad.**

> **Ojo (dos versiones de los cinco pasos).** La slide general lista *Seleccionar, Acordar, Integrar, Medir, Mejorar juntos*, y la del ejemplo *Acordar, Integrar, **Probar**, Medir, Mejorar juntos* (reemplaza "Seleccionar" por "Probar", porque ya hay un proveedor elegido). Si te piden "los pasos", podés mencionar ambas variantes.

9.12. 8.12 Cómo se conectan los ocho principios

No son una lista de compras: se **refuerzan entre sí**. Una forma de verlos (análisis mío):

Grupo	Principios	Función
El norte	1 Cliente, 2 Liderazgo	Para quién trabajamos y quién marca el rumbo
La gente	3 Participación	Quién construye la calidad
La estructura	4 Procesos, 5 SGC	Cómo se organiza el trabajo y qué lo sostiene
El aprendizaje	6 Mejora continua, 7 Evidencia	Cómo se mejora y con qué base se decide
El ecosistema	8 Proveedores	La calidad que viene de afuera

Ejemplos de interdependencia:

- Sin **liderazgo** (2) no hay recursos para la **mejora continua** (6) ni la **participación** (3).
- Sin **evidencia** (7), la **mejora** (6) se convierte en adivinanza; sin **mejora**, la evidencia no se usa para nada.
- Sin **procesos** (4) claros, el **SGC** (5) es solo papel.
- Sin **cliente** (1) como referencia, optimizar procesos (4) puede ser optimizar lo irrelevante.

9.13. 8.13 Cómo detectar qué principio se incumple en un caso

Método práctico (aporte mío; coincide con la lógica de la Guía del TP, donde se pide estado, evidencia, confianza e impacto por principio):

1. **Separá hechos de opiniones.** Subrayá los hechos del caso. (*Una opinión puede ser evidencia de percepción, pero no prueba por sí sola una causa*, dice la Guía del TP.)
2. **Buscá el síntoma** y preguntate qué principio protege contra ese síntoma (tabla abajo).
3. **Citá la evidencia** del caso que sostiene tu respuesta.
4. **No tienen que incumplirse los ocho**: algunos se cumplen o no hay evidencia suficiente.

Si ves…	Pensá en el principio…
Métricas de uso que suben, pero reclamos de que “no encuentran nada”; funciones que nadie pidió	**1. Orientación al cliente**
La dirección exige velocidad pero no da recursos; discurso sin ejemplo; culpa al equipo	**2. Liderazgo**
Ideas del personal que no se escuchan; solo “cumplir tareas”; miedo a señalar problemas	**3. Participación**
Mismo dato cargado dos veces, traspasos manuales, nadie responsable del recorrido completo	**4. Procesos**
La calidad depende de una persona; sin objetivos, indicadores ni documentación vigente	**5. SGC**
Los mismos defectos reaparecen; retrospectivas sin acciones; mejoras que no se sostienen	**6. Mejora continua**
Decisiones por intuición, jerarquía o urgencia; datos sin validar; “culpables” en vez de causas	**7. Evidencia**
Proveedor evaluado solo por precio; sin protocolo de cambios; culpas cruzadas	**8. Proveedores**

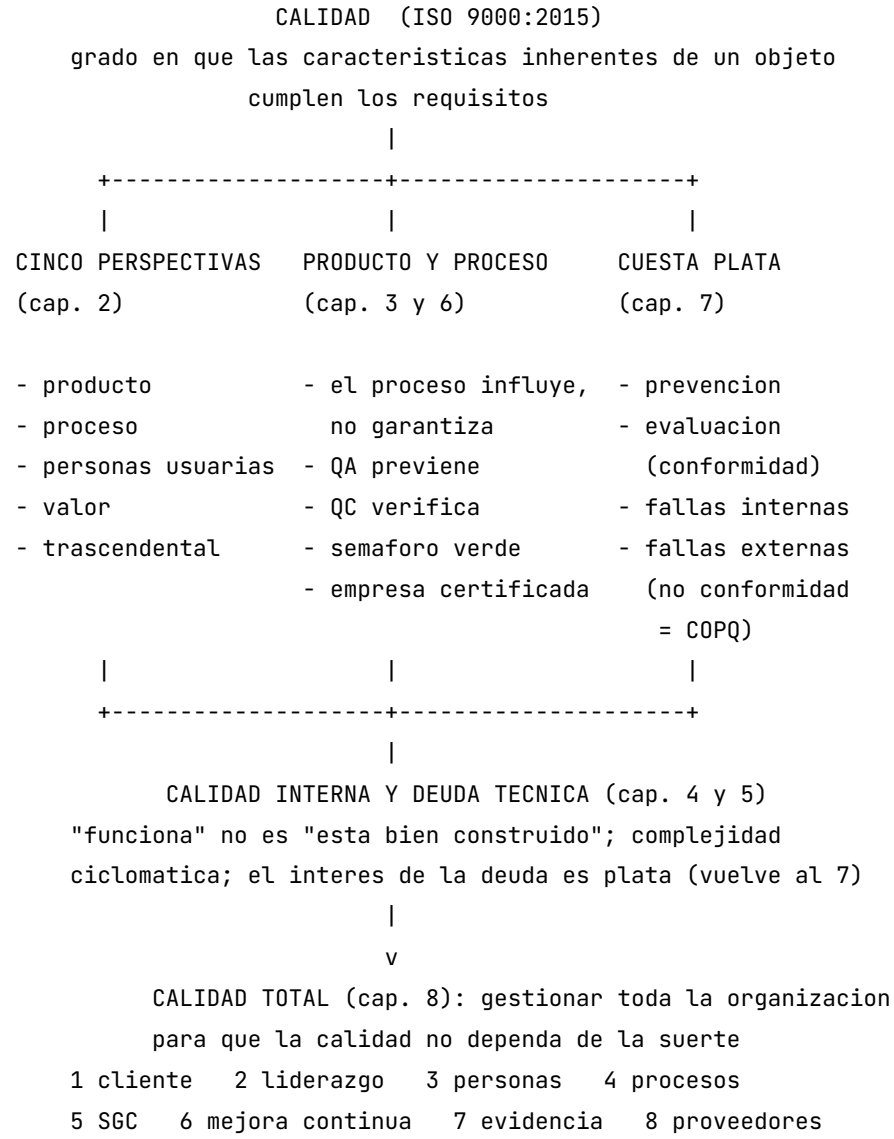
9.14. 8.14 Para recordar

- **Calidad Total** = excelencia que satisface al cliente **interno y externo**, al **menor costo**, en armonía con el **entorno social**, en un **proceso continuo**; objetivo final: **supervivencia y sostenibilidad.**
- **Ocho principios:** 1 Orientación al cliente, 2 Liderazgo, 3 Participación del personal, 4 Enfoque basado en procesos, 5 Enfoque de sistema / SGC, 6 Mejora continua, 7 Decisiones basadas en evidencia, 8 Relación con proveedores.
- **Frases:** (1) *observar si logra su objetivo*; (2) *la calidad se lidera, se facilita y se demuestra*; (3) *participar es aportar, decidir y transformar*; (4) *mejorar el flujo completo*; (5) *SGC ≠ certificar*; (6) *hacer del aprendizaje una práctica de gestión* (PDCA, Kaizen, DMAIC, retrospectivas); (7) *evidencia + criterio experto*; (8) *la falla del proveedor es experiencia del cliente.*

10. 9. Integración: cómo se conectan los temas

Hasta acá estudiaste cada tema por separado. En un parcial teórico-práctico, lo difícil suele ser **reconocer qué tema está en juego** y **combinar varios en una misma respuesta**. Este capítulo sirve para eso.

10.1. 9.1 Mapa conceptual de la unidad



Cómo leerlo. La calidad se define respecto de requisitos (cap. 1) y se mira desde varias perspectivas (cap. 2). Dos de ellas, producto y proceso (cap. 3), se influyen pero no se garantizan mutuamente (cap. 6). Mirar por dentro el producto muestra la calidad interna y la deuda técnica (cap. 4 y 5). Todo termina en plata (cap. 7), y la Calidad Total es la forma de gestionar la organización completa para que eso no dependa de esfuerzos aislados (cap. 8).

10.2. 9.2 Tabla de reflejos: "si te dicen X, pensá en Y"

Una buena forma de resolver rápido una pregunta es reconocer **palabras ancla**. (Tabla armada por mí a partir de las slides.)

Si la pregunta menciona…	Pensá en…	Cap.
"grado", "características inherentes", "requisitos"	Definición ISO 9000:2015	1
"no cuesta", "hacerlo bien la primera vez"	Crosby; la no calidad es lo caro	1, 7
"especificado" **y** "necesidades o expectativas"	Definición IEEE; lo pedido $\neq$ lo necesario	2
"medible", "complejidad", "líneas de código", "defectos"	Perspectiva de **producto**	2
"estándares", "conformidad con procesos", "trazabilidad"	Perspectiva de **proceso**	2
"facilidad de uso", "accesibilidad", "satisfacción"	Perspectiva de **personas usuarias**	2
"equilibrio entre beneficio, costo y restricciones"	Perspectiva de **valor**	2
"excelencia", "difícil de definir"	Perspectiva **trascendental**	2
"influye pero no garantiza", "el proceso crea las condiciones"	Producto vs. proceso	3
"previene", *Definition of Done*, estrategia de testing	**QA**	3
"verifica", "ejecuta pruebas", "detecta defectos"	**QC**	3
"funciona pero cambiarlo es caro y riesgoso", "17 lugares", "600 líneas"	Calidad **interna** / deuda técnica	4, 5
"caminos independientes", "decisiones + 1", "casos mínimos"	**Complejidad ciclomática**	4
"interés", "atajo", "se paga mañana"	**Deuda técnica**	5
"iceberg", "confianza", "reputación", "desgaste"	Costos **invisibles**	5
"todo en verde pero los usuarios se quejan"	**Semáforo verde** (proceso $\neq$ producto)	6
"certificada", "auditoría sin hallazgos"	La certificación respalda al **proceso**	6
capacitación, estándares, revisión de requisitos	Costo de **prevención**	7

Si la pregunta menciona…	Pensá en…	Cap.
pruebas planificadas, auditorías, análisis estático	Costo de **evaluación**	7
retrabajo o corrección **antes** de entregar	**Falla interna**	7
incidentes, soporte, hotfix, clientes perdidos	**Falla externa**	7
“COPQ”, “porcentaje de fallas externas”, DRE, *escape rate*	Indicadores económicos	7
“cliente interno y externo”, “supervivencia”	**Calidad Total**	8
“observar si logra su objetivo” / “se lidera, se facilita y se demuestra” / “participar es aportar, decidir y transformar”	Principios 1 / 2 / 3	8
“mejorar el flujo completo” / “SGC $\neq$ certificar” / “hacer del aprendizaje una práctica de gestión”	Principios 4 / 5 / 6	8
“evidencia + criterio experto” / “la falla del proveedor es experiencia del cliente”	Principios 7 / 8	8

10.3. 9.3 Un mismo caso, ocho lecturas

Para ver cómo los capítulos se apoyan entre sí, tomemos **un solo caso inventado por mí** y miremos qué dice cada tema.

> **Caso (ejemplo propio).** Una clínica lanza *TurnosYa*, una app para sacar y reprogramar turnos. Hoy funciona: la gente saca turnos todos los días.

Cap.	Lectura del caso
1	¿Calidad **respecto de qué requisitos y de quién**? El paciente quiere simplicidad; recepción, ver la agenda; la clínica, menos ausentismo; la ley de datos personales es una obligación.
2	*Producto:* respuesta en menos de 2 segundos. *Proceso:* cada cambio con revisión y pruebas. *Personas:* una persona mayor reprograma sola. *Valor:* priorizar reprogramación y recordatorios sobre animaciones. *Trascendental:* la app “da confianza”.
3	La *Definition of Done* y la revisión de código son **QA**; las pruebas de regresión antes de cada versión son **QC**.
4	La regla que decide si un turno está libre vive en una función de 400 líneas con complejidad 25. Nadie quiere tocarla; para cubrir sus caminos básicos harían falta al menos 25 casos.

Cap.	Lectura del caso
5	Esa regla está copiada en 9 pantallas. Agregar "turno de telemedicina" estimado en 2 días lleva 2 semanas: la diferencia es el **interés** de la deuda técnica.
6	El tablero del sprint está todo en verde, pero aparecen **turnos duplicados** porque nadie midió el comportamiento con muchos usuarios a la vez.
7	Capacitación (prevención), pruebas (evaluación), retrabajo (falla interna), soporte y pacientes que se van (falla externa).
8	Principio 1: ¿el paciente *logra* sacar su turno? 2: la dirección da tiempo para refactorizar. 3: recepción aporta lo que ve a diario. 4: se mejora el flujo completo de turnos, no solo la app. 6: retrospectivas con acciones. 7: se decide con *logs* y reclamos. 8: acuerdo de servicio (SLA) con quien envía los SMS.

10.4. 9.4 Relaciones entre temas que conviene tener claras

(Síntesis mía; cada relación está apoyada en lo que dicen las slides.)

- **Crosby, costos y Calidad Total.** Crosby dice que lo caro es la no calidad (cap. 1). Los costos de la calidad muestran cuánto (cap. 7). Y la definición de Calidad Total pide excelencia "al **menor costo posible**" (cap. 8): hacerlo bien a la primera no es un lujo, es eficiencia.
- **Deuda técnica y costos.** El "interés" de la deuda aparece, con el tiempo, como **retrabajo** (falla interna) y como **incidentes** (falla externa). Una deuda que no se registra compite de manera invisible con cada funcionalidad nueva (cap. 5).
- **Complejidad, pruebas y costos.** Más complejidad significa más caminos que probar, más difícil de entender y más riesgo de errores al cambiar (cap. 4). Es decir, más evaluación y más fallas.
- **Semáforo verde y evidencia.** El semáforo verde ocurre cuando solo se mide el proceso. La cura son los principios 1 y 7: observar si el usuario logra su objetivo y decidir con evidencia que incluya el producto (cap. 6 y 8).
- **QA/QC y prevención/evaluación.** QA se parece a lo que financia la **prevención**; QC, a lo que financia la **evaluación**. No son idénticos (los costos se clasifican por su naturaleza económica, QA y QC por su foco), pero conviene verlos juntos (aporte mío).

10.5. 9.5 Tres lentes sobre una misma actividad

Una misma actividad se puede mirar desde tres lentes: **QA o QC**, **categoría de costo** y **principio de Calidad Total**. Clasificación mía, para practicar; algunas actividades tienen zonas grises y lo importante es **justificar**.

Actividad	¿QA o QC?	Costo	Principio que más se refleja
Capacitar al equipo en técnicas de testing	QA	Prevención	3 Participación (y 2 Liderazgo, por dar recursos)
Definir una *Definition of Done* con criterios de calidad	QA	Prevención	5 SGC / 4 Procesos
Revisar los requisitos con el cliente antes de programar	QA	Prevención	1 Cliente
Acordar un SLA con el proveedor de pagos	QA	Prevención	8 Proveedores
Ejecutar pruebas de regresión antes de liberar	QC	Evaluación	7 Evidencia
Análisis estático con un límite de complejidad	QC si solo mide; QA si es un *quality gate* que bloquea el avance	Evaluación	7 Evidencia
Corregir y volver a probar un defecto hallado en pruebas de sistema	(es corrección, no verificación)	Falla interna	6 Mejora continua, si se analiza la causa
Atender reclamos por una falla en producción	(consecuencia, no actividad de calidad)	Falla externa	1 Cliente

11. 10. Autoevaluación

11.1. 10.1 Cómo usarla

1. Respondé **sin mirar** el apunte ni el Anexo A; lo que no puedas responder es tu mapa de estudio.
2. Corregí con el **Anexo A** (importa más *por qué* que *cuál*) y volvé a los capítulos de lo que fallaste.

> **Ojo.** Las preguntas, ejercicios y casos de este capítulo son **míos** y **no son del examen** (salvo el ejercicio 6, que reproduce la función del parcial 2025). Las preguntas reales de 2025 están en el Anexo D.

11.2. 10.2 Preguntas de desarrollo

D1. Definí calidad según ISO 9000:2015, explicando cada término clave. ¿Por qué se dice que la calidad es relativa? ¿Significa que es arbitraria?

D2. Explicá la frase de Crosby "la calidad no cuesta". ¿Quiere decir que prevenir es gratis?

D3. Comparé las definiciones de calidad de ISO y de IEEE. ¿Qué hace explícito IEEE? Explicá con un ejemplo propio la diferencia entre *lo que se pide* y *lo que se necesita*.

D4. Nombrá las cinco perspectivas de la calidad del software, con su pregunta clave, y dá un ejemplo de cada una para una app de delivery.

D5. ¿Qué significa que el proceso "influye pero no garantiza" la calidad del producto? Dá un ejemplo.

D6. Diferenciá QA de QC. ¿Por qué se dice que QA no es sinónimo de testing?

D7. El gerente dice: "el sistema funciona perfecto, ¿cuál es el problema?". Respondé con el concepto de calidad interna y explicá por qué el problema se nota recién al modificar el software.

D8. ¿Qué mide la complejidad ciclomática y cómo se calcula de forma práctica? ¿Qué relación tiene con las pruebas? Si dividís una función de complejidad 8 en cuatro más chicas, ¿mejora algo? ¿Qué pasa con la suma de las complejidades?

D9. Definí deuda técnica y explicá la metáfora (préstamo, interés, pago). Explicá el cuadrante de la deuda con un ejemplo en cada celda.

D10. ¿Qué representa el iceberg de costos de la no calidad? Dá tres costos visibles y tres invisibles, y explicá por qué los invisibles son peligrosos para tomar decisiones.

D11. Explicá el caso del "semáforo verde": qué estaba midiendo la organización, qué no, y qué conclusión se desprende.

D12. ¿Qué acredita una certificación de un sistema de gestión de calidad y qué no? ¿Qué error cometió la organización contratante en el caso de la empresa certificada y qué debería haber hecho?

D13. Describí las cuatro categorías de costos de la calidad. Agrupalas en conformidad y no conformidad, definí el COPQ y explicá por qué gastar en testing **no** es un costo de no calidad.

D14. Definí Calidad Total. ¿Qué es un cliente interno? ¿Por qué el objetivo final es la supervivencia de la organización?

D15. Explicá el principio de enfoque basado en procesos usando el caso de transformación digital en turismo. ¿Por qué "transformar digitalmente no es cambiar una herramienta"?

D16. Compará PDCA, Kaizen, Six Sigma (DMAIC) y las retrospectivas ágiles. ¿Cómo elegís cuál usar? Explicá qué aporta DMAIC en el caso de las fallas del checkout.

D17. Principio 7: ¿por qué se dice "evidencia + criterio experto"? ¿Qué rol cumple el diagrama de Ishikawa y por qué "propone hipótesis" y no "encuentra culpables"?

D18. Principio 8: ¿por qué "la falla del proveedor también es experiencia del cliente"? Nombrá tres tipos de proveedores en software y los pasos para gestionarlos.

D19. Integradora. Relacioná **deuda técnica**, **costos de la no calidad** y **Calidad Total**: ¿cómo se conectan los tres conceptos?

11.3. 10.3 Opción múltiple

Marcá **la** respuesta correcta (en el parcial real puede haber más de una; acá hay una sola por pregunta).

M1. Según ISO 9000:2015, la calidad es:

a) La ausencia total de defectos en el producto entregado.

b) El grado en el que un conjunto de características inherentes de un objeto cumple con los requisitos.

c) El cumplimiento del plazo y del presupuesto acordados.

d) El nivel de satisfacción que los usuarios expresan en una encuesta.

M2. La frase de Crosby "la calidad no cuesta" significa principalmente que:

a) Invertir en prevención no requiere esfuerzo ni recursos.

b) Las empresas no deberían medir los costos de la calidad.

c) Lo que realmente cuesta dinero es no hacer bien las cosas la primera vez.

d) El testing no tiene costo si se automatiza.

M3. Según IEEE, la calidad del software es el grado en que se cumplen:

a) Solo los requerimientos especificados.

b) Solo las necesidades que el usuario expresó en la entrevista.

c) Solo los estándares de codificación de la organización.

d) Los requerimientos especificados y también las necesidades o expectativas del cliente o usuario.

M4. "El equipo mide tiempos de respuesta, cantidad de defectos y complejidad del código para decidir si el sistema tiene calidad." Esto corresponde a la perspectiva:

a) De producto.

b) De proceso.

c) Trascendental.

d) De valor.

M5. "Se decide no invertir en animaciones y concentrarse en el flujo de pago, porque es lo que más beneficio aporta dentro del presupuesto y del plazo." Esto corresponde a la perspectiva:

a) Trascendental.

b) De proceso.

c) De producto.

d) Basada en valor.

M6. "Definir criterios de calidad en la *Definition of Done* y una estrategia de testing para evitar defectos" es una actividad de:

a) QC, porque verifica el producto.

b) QA, porque previene defectos desde el proceso.

c) QC, porque incluye la palabra *testing*.

d) Ninguna: es calidad del producto.

M7. ¿Qué frase resume mejor la relación entre calidad del proceso y calidad del producto?

a) Un buen proceso garantiza un buen producto.

b) El producto es independiente del proceso con el que se construyó.

c) El proceso crea las condiciones; el producto confirma la calidad.

d) Un proceso certificado reemplaza las pruebas del producto.

M8. Una función tiene dos `if` (uno de ellos con la condición compuesta `a && b`), un `for` y un `else`. Contando cada `&&` como una decisión (convención de ESLint), su complejidad ciclomática es:

a) 5

b) 4

c) 6

d) 3

M9. Si una función tiene complejidad ciclomática 6, eso implica que:

a) La función tiene 6 líneas.

b) La función tiene 6 defectos.

c) La función debe dividirse sí o sí en 6 funciones.

d) Hacen falta como mínimo 6 casos de prueba para cubrir los caminos básicos.

M10. La deuda técnica es:

a) El dinero gastado en licencias y herramientas de desarrollo.

b) El trabajo adicional futuro que se acumula por haber elegido una solución fácil a corto plazo (deficiencias de calidad interna que encarecen el cambio).

c) Cualquier defecto conocido que aún no se corrigió.

d) La falta de documentación del código fuente.

M11. En la metáfora de la deuda técnica, el "interés" es:

a) El costo de las licencias.

b) La ganancia obtenida por entregar rápido.

c) El esfuerzo adicional que exige cada cambio por la mala calidad interna.

d) El costo de refactorizar.

M12. "Entregamos ahora y planificamos corregir" corresponde, en el cuadrante de la deuda, a la deuda:

a) Moderada y voluntaria.

b) Excesiva y voluntaria.

c) Moderada e involuntaria.

d) Excesiva e involuntaria.

M13. ¿Cuál de los siguientes es un costo **invisible** de la no calidad?

a) Las horas de retrabajo.

b) La pérdida de confianza y el desgaste del equipo.

c) El soporte por incidentes.

d) La corrección de defectos.

M14. Un proyecto con todos los indicadores de gestión en verde que entrega un producto lento y confuso demuestra que:

a) La metodología ágil no sirve.

b) El equipo incumplió el proceso.

c) Cumplir el proceso no garantiza la calidad del producto.

d) Hay que medir más tareas completadas.

M15. Una certificación de un sistema de gestión de calidad acredita que:

a) Cada producto cumple todos sus requisitos.

b) El software entregado no tiene defectos.

c) El rendimiento del software es el esperado.

d) La organización tiene un sistema de gestión definido, implementado y auditado según una norma.

M16. "Retrabajo por un defecto detectado en pruebas de sistema, antes de liberar la versión" es un costo de:

a) Falla interna.

b) Falla externa.

c) Evaluación.

d) Prevención.

M17. El COPQ (costo de la no calidad) se calcula como:

a) Prevención + evaluación.

b) Fallas internas + fallas externas.

c) Conformidad + no conformidad.

d) Evaluación + fallas externas.

M18. Un proyecto detectó 200 defectos en total y 30 de ellos aparecieron en producción. Su eficiencia de remoción de defectos (DRE) es:

a) 15 %

b) 30 %

c) 85 %

d) 70 %

M19. En un proyecto, las fallas externas representan el 64 % del COPQ y la prevención apenas el 11 % del costo total de calidad. La conclusión más razonable es:

a) Prevenir y detectar antes, porque los defectos se están escapando a producción.

b) Agregar más pruebas al final del desarrollo.

c) Reducir el presupuesto de calidad.

d) Esperar a tener más datos antes de hacer cualquier cambio.

M20. Según la definición de la cátedra, el objetivo final de la Calidad Total es:

a) Obtener la mayor cantidad posible de certificaciones.

b) Reducir al mínimo el equipo de calidad.

c) La supervivencia y sostenibilidad de la organización.

d) Eliminar los costos de prevención.

M21. "La dirección exige entregar rápido, no asigna tiempo para mejoras y responsabiliza al equipo cuando hay errores." ¿Qué principio se incumple principalmente?

a) Relación con los proveedores.

b) Enfoque basado en procesos.

c) Toma de decisiones basada en evidencia.

d) Liderazgo y compromiso con la calidad.

M22. Implementar un Sistema de Gestión de la Calidad (SGC) significa:

a) Diseñar una forma sistemática de dirigir y controlar la organización para asegurar y mejorar la calidad; certificarlo puede ser una decisión posterior.

b) Obtener obligatoriamente una certificación ISO 9001.

c) Contratar un equipo de QA.

d) Documentar todo lo que hace la organización, sin importar para qué.

M23. El diagrama de Ishikawa, en el marco de la toma de decisiones basada en evidencia:

a) Sirve para identificar a la persona responsable del error.

b) Mide la complejidad ciclomática del sistema.

c) Organiza causas posibles como hipótesis, que luego la evidencia confirma o descarta.

d) Reemplaza la necesidad de reunir datos.

M24. Las etapas del ciclo PDCA son:

a) Planear, probar, producir, publicar.

b) Planificar, hacer, verificar, actuar.

c) Preparar, diseñar, codificar, auditar.

d) Priorizar, definir, controlar, aprobar.

M25. "Un pago falla porque la pasarela externa no responde, pero el cliente ve que *tu* tienda falla." Esto remite al principio de:

a) Participación del personal.

b) Mejora continua.

c) Liderazgo.

d) Relación con los proveedores.

11.4. 10.4 Verdadero o falso (justificá)

Decí si es verdadero o falso y **explicá por qué en una línea**. Una afirmación falsa casi nunca lo es "por completo": identificá la parte que falla.

VF1. La calidad es subjetiva, por lo tanto no puede medirse.

VF2. Si un software cumple todos los requerimientos especificados, entonces es de calidad según IEEE.

VF3. La perspectiva de proceso y la perspectiva de producto se refieren a lo mismo.

VF4. Un buen proceso de desarrollo garantiza un buen producto.

VF5. QA es sinónimo de testing.

VF6. Una persona con el cargo "QA" que ejecuta casos de prueba manuales está haciendo, conceptualmente, tareas de QC.

VF7. Un sistema que no muestra fallas visibles para el usuario tiene buena calidad interna.

VF8. La mala calidad interna suele hacerse visible recién cuando hay que modificar el software.

VF9. El objetivo ideal es que toda función tenga complejidad ciclomática 1.

VF10. Si dividís una función de complejidad 8 en cuatro funciones más chicas, la suma de las complejidades de las cuatro será menor que 8.

VF11. Con la regla `complexity` de ESLint, el `else` no suma complejidad.

VF12. Gestionar la deuda técnica significa eliminarla por completo.

VF13. El costo de realizar pruebas planificadas es un costo de no calidad.

VF14. Cuando las fallas externas dominan el COPQ, la señal es que hay que prevenir y detectar antes.

VF15. Un indicador aislado alcanza para gestionar la calidad.

VF16. Si la auditoría de una certificación no encuentra incumplimientos, se puede asegurar que el producto cumple sus requisitos.

VF17. Implementar un SGC es lo mismo que certificar la norma ISO 9001.

VF18. En Calidad Total, el cliente puede ser interno (quien recibe mi trabajo dentro de la organización) o externo.

VF19. No toda evidencia es numérica: decidir con evidencia combina los datos con el criterio experto.

VF20. El escape rate y el DRE (calculados sobre el mismo total de defectos) suman 100 %.

VF21. La deuda técnica que no se registra compite de manera invisible con cada funcionalidad nueva.

Categoría	Costo

11.5. 10.5 Ejercicios numéricos de costos

Ejercicio 1. Un proyecto tiene un presupuesto total de **US$ 250.000**. Costos asociados a la calidad:

Categoría	Costo
Prevención	US$ 10.000
Evaluación	US$ 15.000
Fallas internas	US$ 30.000
Fallas externas	US$ 45.000

Calculá:

a) El costo total de la calidad y qué porcentaje es del presupuesto.

b) Los costos de conformidad y de no conformidad (en monto y como % del costo total de la calidad).

c) El COPQ como porcentaje del presupuesto.

d) Qué porcentaje del COPQ son fallas internas y qué porcentaje fallas externas.

e) ¿Qué le recomendarías al equipo y por qué?

Ejercicio 2. Un proyecto detectó **200 defectos** en total, de los cuales **30** se encontraron en producción. El costo total asociado a fallas fue de **US$ 75.000**. Se dedicaron **480 horas** de retrabajo sobre **3.200 horas** totales del proyecto. Calculá: *escape rate*, DRE, costo promedio por defecto y tasa de retrabajo. ¿Qué indicador te preocuparía más y por qué?

Ejercicio 3. Una plataforma SaaS tuvo en el año **ingresos de US$ 800.000**, **fallas internas de US$ 20.000**, **fallas externas de US$ 60.000** y **16.000 clientes activos**. Calculá el COPQ, el COPQ sobre ingresos, el origen del COPQ (% internas y % externas) y el costo por cliente. ¿Qué lectura hacés?

Ejercicio 4. Clasificá cada costo como **prevención (P)**, **evaluación (E)**, **falla interna (FI)** o **falla externa (FE)**, y decí si es de **conformidad** o de **no conformidad**:

1. Curso de testing para el equipo de desarrollo.
2. Revisión de los requisitos con el cliente antes de empezar a programar.
3. Ejecución de las pruebas de regresión planificadas antes de cada versión.
4. Análisis estático del código en cada integración.
5. Corregir y volver a probar un defecto hallado en las pruebas de sistema.
6. Descartar un módulo ya construido porque no cumplía los requisitos (detectado antes de entregar).

7. Hotfix de urgencia por un error que reportaron los usuarios.
8. El equipo de soporte atendiendo reclamos por un error en producción.
9. Compensación económica a los clientes por una caída del servicio.
10. Auditoría de calidad interna del proyecto.
11. Definir guías de codificación y diseñar la arquitectura antes de implementar.
12. Pérdida de clientes por mala reputación del producto.

11.6. 10.6 Ejercicios de complejidad ciclomática

Ejercicio 5. Dada la función:

```
function clasificarPedido(pedido) {
  let nivel = "normal";
  if (pedido.total > 50000) {
    nivel = "alto";
  } else if (pedido.total > 10000) {
    nivel = "medio";
  }
  for (const item of pedido.items) {
    if (item.agotado || item.cantidad > 100) {
      nivel = "revisar";
    }
  }
  return pedido.urgente ? nivel + "-urgente" : nivel;
}
```

a) Calculá su complejidad ciclomática (contando cada `||` como una decisión).

b) ¿Cuántos casos de prueba hacen falta como mínimo para cubrir los caminos básicos?

c) Armá la tabla de casos con el método "camino base y cambiar una decisión por vez".

Ejercicio 6 (basado en el parcial 2025). Dada la función en Python:

```
def costo_envio(peso: int, urgente: bool, zona: str) -> int:
    # Pre: peso > 0; zona en {"Centro", "Patagonia"}
    if peso ⩽ 5:
        costo = 100
    elif peso ⩽ 20:
        costo = 200
    else:
        costo = 400
    if urgente:
        costo = int(costo * 1.5)
```

```
    if (zona == "Patagonia" and peso > 10):
        costo += 100
    return costo
```

a) Calculá la complejidad de las **dos** formas (con el `and` como decisión aparte y como parte de una sola condición).

b) Armá la tabla de casos de prueba para los caminos básicos (con el `and` contado aparte), indicando la entrada y el resultado esperado.

> **Para el examen.** Ese ejercicio apareció en el parcial 2025 pidiendo "casos de prueba que cubran los caminos básicos y expongan la tabla de casos". Aclará siempre qué convención usás para el `and`.

11.7. 10.7 Casos para analizar

Caso A: "Funciona, no lo toquemos". Un banco calcula los intereses de los préstamos con una regla que está **copiada en 9 servicios distintos**, cada uno con pequeñas diferencias. No hay pruebas automáticas. El área comercial pide cambiar una tasa; el equipo estimó **1 día** y terminó llevando **3 semanas**, con dos errores en producción. El gerente dice: *"El sistema funciona desde hace años, no hay que tocar nada."*

a) ¿Qué conceptos de la unidad están en juego?

b) ¿Dónde está el "interés" de la deuda?

c) ¿En qué cuadrante ubicarías esta deuda? Justificá.

d) Dá dos costos visibles y dos invisibles.

e) Proponé cómo gestionarla (los cuatro pasos).

Caso B: "Quality Gate: Passed". Un equipo muestra a la dirección el panel de SonarQube: *Quality Gate Passed*, mantenibilidad **A**. Pero el mismo panel indica confiabilidad **E** (1.349 *bugs*), **78,8 %** de duplicación y **46 %** de cobertura; la puerta de calidad mira solo el **código nuevo**. La dirección concluye: *"Está aprobado, estamos bien."*

a) ¿Qué significa realmente "Passed" en este caso?

b) ¿Con qué capítulo de la unidad se conecta y qué enseñanza deja?

c) ¿Qué mirarías además del *Quality Gate*?

Caso C: "Retrospectivas sin acciones". Una startup hace una retrospectiva cada sprint y anota los problemas en un tablero, pero nunca asigna acciones ni responsables, y **los mismos problemas reaparecen** (por ejemplo, despliegues que fallan los viernes). Se cuentan los *bugs* reportados, pero nadie analiza sus causas. El CTO dice: *"La calidad es responsabilidad del equipo de QA."*

a) ¿Qué principios de Calidad Total se incumplen? Citá la evidencia del caso.

b) ¿Qué error conceptual comete el CTO respecto de QA y QC?

c) ¿Qué modelo de mejora continua aplicarías y cómo?

12. 11. Errores y trampas frecuentes

12.1. 11.1 Confusiones clásicas

#	Error frecuente	Por qué está mal	Cómo evitarlo
1	"Calidad = sin defectos"	La calidad es **grado de cumplimiento de requisitos**, no perfección	Definir con ISO: grado, características inherentes, requisitos
2	"La calidad es subjetiva, entonces vale cualquier opinión"	Los requisitos **cambian según quién los plantea**, pero una vez explícitos se mide el grado	Preguntar siempre "¿respecto de qué requisitos y de quién?"
3	"Si cumple lo especificado, es de calidad"	IEEE agrega **necesidades y expectativas**	Distinguir lo pedido de lo necesario
4	Mezclar "perspectiva de producto" (medible) con "calidad del producto" (resultado completo)	Son dos usos de la palabra en dos presentaciones	En "¿qué describe la perspectiva basada en el producto?" responder: propiedades **medibles**
5	"Un buen proceso garantiza un buen producto"	Influye, **no garantiza**	Frase: el proceso crea las condiciones; el producto confirma la calidad
6	"QA es testing" o "QA es la persona con ese cargo"	QA previene en el proceso; QC verifica en el producto; el cargo no define la tarea	Preguntar: ¿busca que el defecto no aparezca (QA) o encontrar el que ya existe (QC)?
7	"Funciona, entonces tiene calidad"	La calidad interna se nota **al cambiar**	Pensar en mantenibilidad y costo del próximo cambio
8	En complejidad: contar el `else` o el `default`, o **olvidar el +1**	`else` y `default` no suman; la base es 1	Contar decisiones y sumar 1 al final
9	Olvidar los operadores lógicos (Y, O) de las condiciones compuestas, o no aclarar la convención	Según la herramienta, el resultado cambia	Escribir "cuento cada operador lógico como una decisión"
10	"Dividir la función reduce la complejidad total"	Reparte las decisiones; cada unidad mejora, la suma puede aumentar	Hablar de unidades **simples, testeables y comprensibles**

#	Error frecuente	Por qué está mal	Cómo evitarlo
11	"Deuda técnica = bug / licencias / falta de documentación"	Es trabajo futuro adicional por **deficiencias de calidad interna**	Conectar con "interés" y "atajo"
12	"Toda la deuda hay que eliminarla"	Se gestiona: decisiones conscientes y que no crezca sin control	Recordar los cuatro pasos y el cuadrante
13	"El testing es un costo de no calidad"	Es **evaluación** (conformidad). El costo de no calidad es corregir lo que se escapa	COPQ = fallas internas + externas
14	Confundir "costos de la calidad" (total) con "costo de la no calidad" (solo fallas)	Son conjuntos distintos	Total = conformidad + no conformidad
15	Calcular un porcentaje con el **denominador equivocado**	Ingresos, presupuesto, costo de calidad y COPQ dan resultados distintos	Escribir la fórmula antes de calcular y marcar el denominador
16	Invertir DRE y *escape rate*	DRE es lo que se **eliminó antes de producción**; el escape rate, lo que **llegó**	DRE alto es bueno; escape rate alto es malo
17	"Con un indicador alcanza"	Una señal aislada no dice si mejorás; la **evolución** sí	Comparar en el tiempo
18	"Implementar un SGC es certificar ISO 9001"	La certificación puede ser una decisión posterior	Frase: un SGC convierte la calidad en una capacidad de la organización
19	"Empresa certificada, entonces producto de calidad"	La certificación respalda al **proceso**	Pedir evidencia de producto: pruebas, métricas, aceptación
20	"Cliente = quien paga"	En Calidad Total hay clientes **internos y externos**	Pensar en quién recibe mi trabajo
21	"Evidencia = solo números"	Evidencia + criterio experto; no toda evidencia es numérica	Combinar datos con interpretación
22	Ishikawa para "buscar culpables"	Organiza hipótesis; la evidencia las confirma o descarta	Frase: ayuda a comprender el sistema que produce el problema

23	Ponerle otro nombre al principio 5	Figura como “Enfoque de sistema” y como “Implementación de un SGC”	Escribir “Enfoque de sistema / Implementación de un SGC”

12.2. 11.2 Trampas de redacción en preguntas de opción múltiple

- **“Marcar la/s respuesta/s correcta/s”** significa que **puede haber más de una** (en el parcial 2025, la pregunta 4 de costos tenía las tres correctas).
- Desconfiá de las palabras absolutas (**“únicamente”, “siempre”, “garantiza”, “solo”, “nunca”**): casi nada es absoluto en esta unidad. Si dos opciones parecen correctas, elegí la que **usa la formulación de la cátedra**.
- Los distractores suelen ser **la respuesta de otro tema**: preguntate de qué tema es cada uno.

12.3. 11.3 Cómo responder un caso en el parcial

1. **Identificá el concepto** que el caso ilustra (¿semáforo verde? ¿deuda técnica? ¿principio 4?).
2. **Citá evidencia del enunciado**, con sus palabras: sin evidencia, la respuesta vale menos.
3. **Concluí con la idea de la unidad** en una frase (por ejemplo, “cumplir el proceso no garantiza la calidad”).
4. **Proponé una acción concreta** (qué medir, qué cambiar) y **dá el matiz** si corresponde (“no hay evidencia suficiente”).

Antes de entregar: ¿definí los términos, marqué el denominador y aclaré la convención de complejidad? ¿Dije qué concepto o principio es y por qué?

13. Anexo A. Respuestas de la autoevaluación

> **Ojo.** Son **respuestas modelo mías**, armadas con lo que dicen las slides. En un parcial real alcanza con que tu respuesta tenga las ideas centrales; no hace falta que se parezca palabra por palabra.

13.1. A.1 Preguntas de desarrollo

D1. Calidad (ISO 9000:2015). Calidad es el **grado** en que un conjunto de **características inherentes** de un **objeto** cumple con los **requisitos**. *Objeto*: producto, servicio o proceso. *Características inherentes*: lo que el objeto es y hace (rendimiento, facilidad de uso, seguridad). *Requisitos*: necesidades, expectativas y obligaciones. *Grado*: es una medida de cumplimiento, no un sí/no. Es **relativa** porque los requisitos dependen de quién los plantea (la hamburguesa abundante o la saludable: la misma pieza puede ser excelente para una persona e inadecuada para otra). **No es arbitraria**: una vez explícitos los requisitos, se puede medir el grado de cumplimiento; por eso la ingeniería de requisitos es tan importante.

D2. Crosby. *"La calidad no cuesta"* quiere decir que lo que realmente cuesta dinero son las cosas que no tienen calidad: todas las acciones (retrabajo, correcciones, reclamos, soporte, clientes perdidos) que resultan de **no hacer bien las cosas la primera vez**. **No** significa que prevenir sea gratis (*"no es un regalo"*: requiere esfuerzo e inversión), sino que lo que se invierte en prevenir es menos que lo que se evita gastar en corregir. Es el puente hacia los costos de la calidad (cap. 7).

D3. ISO vs. IEEE. ISO es general: evalúa cualquier objeto contra los *requisitos* (que ya incluyen necesidades, expectativas y obligaciones). IEEE es específica de software y **separa dos cosas**: los requerimientos especificados **y** las necesidades o expectativas del cliente o usuario. Hace explícito que **cumplir lo escrito no alcanza**. *Ejemplo propio:* el área comercial pide "un listado de ventas exportable a Excel" y el sistema lo entrega perfecto (lo pedido). Pero necesitaba decidir qué mercadería reponer, y el listado no lo permite (lo necesario). Cumplió el requerimiento y falló la necesidad.

D4. Las cinco perspectivas (ejemplos para una app de delivery).

Perspectiva	Pregunta clave	Ejemplo
Producto	¿Qué propiedades tiene?	El listado de comercios carga en menos de 2 segundos y no hay defectos críticos abiertos
Proceso	¿Se construyó correctamente?	Cada cambio pasa por revisión de código, pruebas automáticas y trazabilidad al requerimiento
Personas usuarias	¿Le sirve a quien la usa?	Una persona con poca experiencia puede pedir y pagar sin ayuda, también con lector de pantalla
Valor	¿Qué nivel de calidad aporta mayor valor?	Con el presupuesto y plazo fijos, se prioriza el pago confiable y el seguimiento del pedido sobre funciones decorativas

Perspectiva	Pregunta clave	Ejemplo
Trascendental	¿Qué la hace excelente?	La app "se siente sólida": coherente, confiable, nunca falla a la hora de la cena

D5. "Influye pero no garantiza". Un mal proceso **aumenta el riesgo** de errores, retrabajo y resultados inconsistentes; un buen proceso lo **reduce**, pero por sí solo no asegura un buen producto, porque la calidad del producto se **confirma evaluando el producto** (pruebas y métricas). *Ejemplo:* el caso del semáforo verde: proceso impecable, producto lento y confuso. (Analogía: seguir una receta hace probable que el plato salga bien; la única forma de saberlo es probarlo.)

D6. QA vs. QC. QA *previene*: se enfoca en el **proceso** y las prácticas (*Definition of Done*, estándares, *code review*, *quality gates*, estrategia de testing, CI/CD) y pregunta *¿cómo evitamos que aparezcan defectos?* **QC** *verifica*: se enfoca en el **producto** (pruebas funcionales, de regresión, de performance, de seguridad, exploratorias) y pregunta *¿el producto tiene defectos?* **QA no es sinónimo de testing**: ejecutar pruebas es una actividad de QC que aporta evidencia; QA construye las condiciones para que la calidad sea posible, y en *Quality Engineering* la calidad es responsabilidad de todo el equipo.

D7. "Funciona perfecto". Hay que distinguir la **calidad externa** (lo que el usuario ve: resultados correctos) de la **calidad interna** (cómo está construido: claro, modular, comprensible, fácil de modificar). Un software puede dar los resultados esperados y estar tan enredado que cada cambio sea lento, riesgoso y caro. Mientras nadie lo toque, el problema es **invisible**; aparece al **cambiar**: regresiones, estimaciones que se multiplican (de horas a seis días en el caso de los 17 lugares), miedo a tocar ciertas zonas. La calidad interna reduce errores al cambiar, facilita el mantenimiento, baja tiempos y costos y permite que el software evolucione; si no se cumple, el requisito de poder evolucionar (mantenibilidad) queda incumplido.

D8. Complejidad ciclomática. Estima la cantidad de **caminos linealmente independientes** del código; en la práctica, **decisiones + 1** (cuentan `if`, `else if`, bucles, cada `case`, operadores lógicos, ternarios y `catch`; no cuentan `else` ni `default`). Se relaciona con las pruebas porque indica el **número mínimo de casos** para cubrir los caminos básicos (caja blanca). Más complejidad significa más difícil de comprender, probar, modificar y mantener. Dividir una función de complejidad 8 en cuatro **no hace desaparecer las decisiones, las reparte**: cada función nueva suma su propio "+1" y la suma total puede ser mayor (en el ejemplo del apunte: 2 + 3 + 3 + 3 + 1 = 12). Lo que mejora es que cada unidad es **simple, comprensible y se prueba por separado**. El objetivo no es llegar a 1.

D9. Deuda técnica. Son **deficiencias acumuladas en la calidad interna** que hacen más costoso modificar, mantener y evolucionar el software. Metáfora: *pedir prestado* = tomar un atajo (copiar y pegar, saltear diseño o pruebas); *interés* = esfuerzo adicional que exige cada cambio en esa zona; *pagar* = refactorizar, probar, rediseñar; *no pagar* = el interés crece. **Cuadrante** (eje voluntaria/involuntaria × eje excesiva/moderada):

	Voluntaria	Involuntaria
Excesiva / arriesgada	"No hay tiempo para diseñar": el equipo sabe que copiar la regla en 10 lugares es malo y lo hace igual, sin plan	"¿Qué es arquitectura?": un equipo inexperto arma un método de 600 líneas sin saber que debía separar responsabilidades
Moderada / menos arriesgada	"Entregamos ahora y planificamos corregir": se lanza una versión simple para validar y se agenda el refactor	"Ahora sabemos cómo hacerlo mejor": al terminar se entiende un diseño mejor y se registra para mejorar

Lo que importa: **cómo se genera, cuánto riesgo implica y si hay un plan para saldarla.**

D10. Iceberg. Solo se ve la punta: los costos **visibles** (corrección de defectos, retrabajo, soporte e incidentes, reclamos y garantías, demoras inmediatas). Debajo del agua están los **invisibles** (pérdida de confianza, daño reputacional, menor productividad, desgaste del equipo, oportunidades perdidas, mayor costo de evolución, decisiones demoradas). Son peligrosos porque, si se decide solo con lo que se mide, se **subestima** el costo real de la no calidad y se recorta justamente en prevención. La pregunta que deja la slide: *¿qué costos estamos midiendo y cuáles permanecen ocultos?*

D11. Semáforo verde. La organización medía **cumplimiento del proceso y de la gestión**: tareas completadas (95 %), reuniones y retrospectivas, requerimientos en Jira, plazos, presupuesto, riesgos, documentación. **No** medía la calidad del **producto** ni la **experiencia de usuario**: tiempos de respuesta (8 a 12 segundos), comportamiento bajo carga, usabilidad, accesibilidad, uso en móviles, defectos, satisfacción. Las pruebas eran superficiales y al final de cada sprint. Conclusión: **cumplir el proceso no garantiza la calidad**; se gestionó el proyecto, no la calidad del producto. Hace falta QA (prevenir en el proceso) *y* QC (verificar en el producto), con decisiones basadas en evidencia del producto.

D12. Certificación. Acredita que la organización tiene un **sistema de gestión de calidad definido, implementado y auditado** según una norma: capacidad de gestión de procesos. **No** evalúa cada funcionalidad, el rendimiento, la usabilidad o la confiabilidad del software entregado; la auditoría verifica procedimientos y registros, no ejecuta ni mide el software. El error del contratante fue **tomar evidencia de proceso como garantía de producto** y no definir ni verificar requisitos de calidad del producto. Debió exigir **criterios de aceptación medibles** (rendimiento, disponibilidad, compatibilidad móvil, accesibilidad), pruebas de sistema, carga e integración, pruebas de usabilidad con usuarios reales, pilotos y aceptación formal. Frase: *la certificación respalda al proceso; la calidad del producto se verifica en el producto.*

D13. Costos de la calidad.

Categoría	Qué es	Ejemplos
Prevención	Evitar que los defectos aparezcan	Capacitación, estándares, diseño de arquitectura, revisión temprana de requisitos
Evaluación	Comprobar que el producto cumple	Testing, auditorías, análisis estático, automatización
Fallas internas	Defectos detectados **antes** de entregar	Retrabajo, correcciones, demoras
Fallas externas	Defectos detectados **por los usuarios**	Incidentes, soporte, hotfixes, indisponibilidad, compensaciones, pérdida de clientes

Prevención + evaluación = costos de **conformidad** (inversión planificada). Fallas internas + externas = costos de **no conformidad** (reactivos), que es el **COPQ** (*Cost of Poor Quality*). Costo total de la calidad = conformidad + no conformidad. El testing **no** es un costo de no calidad porque es una inversión planificada para detectar defectos (evaluación); lo que cuesta por no tener calidad es **corregir** lo que el testing encontró (falla interna) o lo que se escapó a producción (falla externa).

D14. Calidad Total. Es la **excelencia** en productos o servicios que satisface las expectativas del cliente **interno y externo**, con el **menor costo posible**, en armonía con el **entorno social**, dentro de un **proceso continuo**, porque las expectativas son cambiantes y cada vez más exigentes. **Cliente interno**: quien recibe el trabajo de otra persona dentro de la organización (testing recibe el trabajo de desarrollo). El **objetivo final es la supervivencia** (y sostenibilidad) de la organización porque, si las expectativas suben y la organización no mejora, deja de ser competitiva.

D15. Principio 4 (turismo). *Antes*: las reservas se registraban en Excel y se cargaban a mano en otro sistema para ventas, lo que causaba duplicación de tareas, errores, demoras y pérdida de competitividad. *Después*: se rediseñó el flujo completo (reserva → información única → venta). Por eso "transformar digitalmente no es cambiar una herramienta": **es repensar el proceso de punta a punta**. No alcanza con optimizar cada tarea por separado, hay que mejorar el flujo completo, y *antes de automatizar, entender, modelar y mejorar el proceso.*

D16. Modelos de mejora continua.

Modelo	Para qué sirve
PDCA (Planificar, Hacer, Verificar, Actuar)	Probar cambios, evaluar resultados y estandarizar lo que funciona
Kaizen	Cultura de pequeñas mejoras diarias, con participación de todas las personas

Modelo	Para qué sirve
Six Sigma / DMAIC (Definir, Medir, Analizar, Mejorar, Controlar)	Problemas complejos con variación, defectos y evidencia cuantitativa
Retrospectivas ágiles (Observar, Reflexionar, Acordar, Experimentar)	Que el equipo revise periódicamente cómo trabaja y acuerde mejoras

Se elige según el **problema**, la **evidencia disponible**, el **alcance** y la **velocidad de aprendizaje** necesaria. *En el checkout:* se definió el problema (8 incidentes cada 100 despliegues), se midió durante 12 semanas, se halló que el 70 % de las fallas aparecía cuando los cambios en pagos llegaban tarde a las pruebas integrales, se mejoró (contratos de API, pruebas automáticas del flujo crítico y una regla de despliegue) y se controló con un tablero semanal; el resultado fue pasar de 8 a 2 incidentes (-75 %). Lo que aporta DMAIC es corregir el **patrón del proceso** que genera las fallas, no solo el último *bug*.

D17. Evidencia + criterio experto. Los datos **no reemplazan el criterio: lo hacen más sólido**. Decidir con evidencia es usar información pertinente, confiable y analizada en contexto **junto con** el conocimiento experto; no todo lo medible es importante ni toda evidencia es numérica. El **Ishikawa** organiza las causas posibles por categorías (personas, procesos, tecnología, datos, entorno, medición); *propone hipótesis* y la **evidencia las confirma o descarta**. No busca culpables: ayuda a comprender el sistema que produce el problema.

D18. Proveedores. Porque el cliente **no distingue quién tuvo la culpa**: ve que *tu* producto falla. Un producto es tan confiable como el ecosistema que lo sostiene. Proveedores típicos: nube, APIs externas, pasarelas de pago, servicios de identidad, bibliotecas de código abierto, ciberseguridad, soporte técnico. Pasos: **seleccionar** (capacidad, seguridad, continuidad), **acordar** (SLA, criterios de calidad, responsabilidades), **integrar** (compatibilidad, datos, contingencias), **medir** (disponibilidad, defectos, respuesta) y **mejorar juntos** (en el ejemplo de la tienda, el paso "seleccionar" se reemplaza por "probar").

D19. Integradora (respuesta modelo, aporte mío). La **deuda técnica** es cómo se acumula la no calidad *dentro* del código: una decisión apresurada hoy se paga mañana con intereses. Los **costos de la no calidad** (fallas internas, externas e invisibles) son cómo se ve ese interés *en plata*. La **Calidad Total** es la respuesta de gestión: el liderazgo (2) da recursos para prevenir, la evidencia (7) hace visibles la deuda y los costos, la mejora continua (6) los paga y los procesos y el SGC (4 y 5) evitan que vuelvan a generarse. Todo conecta con Crosby: lo caro no es hacerlo bien, es no hacerlo bien a la primera, y con la definición de Calidad Total, que pide excelencia "al menor costo posible".

13.2. A.2 Opción múltiple

N.º	Resp.	Explicación
M1	**b**	Es la definición ISO. (a) pide perfección, pero la definición habla de *grado*; (c) es gestión del proyecto; (d) la satisfacción es una forma de medir, no la definición.
M2	**c**	Lo caro es la no calidad. (a) contradice "no es un regalo"; (b) es lo opuesto: medir permite decidir; (d) no tiene relación.
M3	**d**	IEEE separa requerimientos especificados **y** necesidades o expectativas. Las otras opciones son parciales.
M4	**a**	Producto = propiedades **medibles** (rendimiento, defectos, complejidad).
M5	**d**	Valor = equilibrio entre beneficio, costo y restricciones.
M6	**b**	Definir criterios en la DoD y una estrategia de testing *previene* desde el proceso: QA. La palabra "testing" no convierte la actividad en QC.
M7	**c**	Es la frase de la cátedra. (a) y (d) afirman garantía; (b) niega la influencia.
M8	**a**	Decisiones: `if` (1) + `&&` (2) + `if` (3) + `for` (4); el `else` no suma. Complejidad = 4 + 1 = **5**.
M9	**d**	Complejidad = caminos independientes = mínimo de casos para cubrir caminos básicos. No es cantidad de líneas ni de defectos, y no obliga a dividir.
M10	**b**	Es trabajo futuro adicional por deficiencias de calidad interna. (a) son licencias; (c) un defecto conocido no es deuda por sí mismo; (d) la falta de documentación puede ser una señal, pero no define la deuda.
M11	**c**	Interés = esfuerzo extra de cada cambio. (d) refactorizar es *pagar* la deuda.
M12	**a**	Decisión consciente, riesgo acotado, con plan de pago: voluntaria y moderada.
M13	**b**	Confianza y desgaste son difíciles de medir y atribuir a un defecto. Las otras tres son costos visibles.
M14	**c**	Es la tesis del Aspecto 03. (b) no: cumplió el proceso, y ese es el punto.
M15	**d**	Acredita un sistema de gestión, no cada producto.
M16	**a**	Defecto hallado **antes** de entregar y corregido: falla interna.
M17	**b**	COPQ = fallas internas + fallas externas. (a) es conformidad; (c) es el total.
M18	**c**	DRE = (200 - 30) / 200 = 170 / 200 = **85 %**. El 15 % de (a) es el escape rate.
M19	**a**	Los defectos se escapan a producción y la prevención recibe poco: "la mayor oportunidad no está en evaluar más, sino en prevenir y detectar antes".
M20	**c**	Objetivo final: supervivencia y sostenibilidad.
M21	**d**	Discurso sin ejemplo, falta de recursos y cultura de culpa: principio 2.
M22	**a**	Implementar un SGC no significa necesariamente certificar.

N.º	Resp.	Explicación
M23	**c**	El diagrama propone hipótesis; la evidencia las confirma o descarta. No busca culpables.
M24	**b**	Planificar, hacer, verificar, actuar (en algunas slides, la última fase se llama “Mejorar”).
M25	**d**	La falla del proveedor se convierte en experiencia del cliente: principio 8.

13.3. A.3 Verdadero o falso

N.º	V/F	Justificación
VF1	**F**	Es *relativa* a los requisitos de quien la plantea, no inmedible: con requisitos explícitos se mide el grado de cumplimiento.
VF2	**F**	IEEE exige además responder a las necesidades o expectativas del cliente o usuario.
VF3	**F**	Producto = propiedades medibles; proceso = cómo se desarrolla (estándares, revisiones, trazabilidad).
VF4	**F**	El proceso influye, pero no garantiza: el producto se confirma evaluándolo.
VF5	**F**	QA previene en el proceso; el testing es una actividad de QC (evidencia sobre el producto).
VF6	**V**	Ejecutar casos de prueba es verificar el producto, aunque el cargo diga “QA”.
VF7	**F**	Puede funcionar y estar mal construido (caso de los 17 lugares y las 600 líneas).
VF8	**V**	Se nota al cambiar: retrasos, errores nuevos y miedo a tocar el código.
VF9	**F**	El objetivo es mantener cada unidad *simple, testeable y comprensible*, no llegar siempre a 1.
VF10	**F**	Cada función nueva suma su “+1”: la suma puede ser mayor (en el ejemplo, 12 contra 8).
VF11	**V**	El `else` no es una decisión nueva (aporta 0).
VF12	**F**	Gestionarla no es eliminarla: es tomar decisiones conscientes y evitar que crezca sin control.
VF13	**F**	Las pruebas planificadas son *evaluación*, un costo de **conformidad**.
VF14	**V**	Es el mensaje del caso Ágora: prevenir y detectar antes.
VF15	**F**	“Un indicador aislado describe una señal; su evolución en el tiempo permite gestionar la calidad.”
VF16	**F**	La auditoría verifica procedimientos y registros; no evalúa el producto entregado.
VF17	**F**	Implementar un SGC no significa necesariamente certificar una norma.

N.º	V/F	Justificación
VF18	**V**	En Calidad Total hay clientes internos y externos.
VF19	**V**	Evidencia + criterio experto; no toda evidencia es numérica.
VF20	**V**	Ambos indicadores usan el mismo denominador (total de defectos detectados), así que son complementarios (observación mía).
VF21	**V**	Es uno de los dos mensajes finales de la slide de gestión de la deuda.

13.4. A.4 Ejercicios de costos

13.4.1. Ejercicio 1

Pregunta	Cálculo	Resultado
a) Costo total de la calidad	10.000 + 15.000 + 30.000 + 45.000	**US$ 100.000**
… como % del presupuesto	100.000 / 250.000	**40 %**
b) Conformidad	10.000 + 15.000	**US$ 25.000** (25 % del costo de calidad)
… No conformidad (COPQ)	30.000 + 45.000	**US$ 75.000** (75 % del costo de calidad)
c) COPQ / presupuesto	75.000 / 250.000	**30 %**
d) Fallas internas / COPQ	30.000 / 75.000	**40 %**
… Fallas externas / COPQ	45.000 / 75.000	**60 %**

e) Recomendación (respuesta modelo). Las fallas externas son el **60 % del COPQ** y la prevención apenas el **10 %** del costo total de calidad (10.000 / 100.000). Además, la no conformidad es **3 veces** la conformidad (75.000 / 25.000): por cada dólar invertido en prevenir y evaluar se gastan US$ 3 en corregir y recuperar. La señal es que los defectos se escapan a producción: conviene **reforzar la prevención y la detección temprana** (requisitos, diseño, capacitación, pruebas desde el inicio), más que sumar pruebas al final.

13.4.2. Ejercicio 2

Indicador	Cálculo	Resultado
Escape rate	30 / 200 x 100	**15 %**

Indicador	Cálculo	Resultado
DRE	(200 - 30) / 200 x 100	**85 %**
Costo promedio por defecto	75.000 / 200	**US$ 375**
Tasa de retrabajo	480 / 3.200 x 100	**15 %**

¿Cuál preocupa más? No hay una única respuesta correcta; lo importante es justificar. Un buen argumento: el *escape rate* mide **lo que llegó al usuario**, que es lo más caro (falla externa), así que 3 de cada 20 defectos escapados merece atención. Otro: el 15 % de retrabajo significa que casi una de cada siete horas se gasta corrigiendo trabajo ya hecho. Y la advertencia de la cátedra: **un indicador aislado describe una señal**; sin la evolución en el tiempo o un objetivo, no se puede decir si 15 % es alto o bajo.

13.4.3. Ejercicio 3

Concepto	Cálculo	Resultado
COPQ	20.000 + 60.000	**US$ 80.000**
COPQ sobre ingresos	80.000 / 800.000 x 100	**10 %**
Origen: fallas internas	20.000 / 80.000 x 100	**25 %**
Origen: fallas externas	60.000 / 80.000 x 100	**75 %**
Costo por cliente	80.000 / 16.000	**US$ 5**

Lectura (aporte mío). De cada US$ 100 que ingresan, US$ 10 se pierden por fallas, y **tres cuartas partes** del COPQ ocurren en producción: los defectos se están escapando a los usuarios. Comparado con el ejemplo de la slide (8 %, 70 % y US$ 4), si fuera la misma empresa en otro año habría **empeorado** en los tres indicadores.

13.4.4. Ejercicio 4

N.º	Costo	Categoría	Bloque
1	Curso de testing para el equipo	**Prevención**	Conformidad
2	Revisión de requisitos con el cliente antes de programar	**Prevención**	Conformidad

N.º	Costo	Categoría	Bloque
3	Pruebas de regresión planificadas antes de cada versión	**Evaluación**	Conformidad
4	Análisis estático del código en cada integración	**Evaluación**	Conformidad
5	Corregir y volver a probar un defecto hallado en pruebas de sistema	**Falla interna**	No conformidad
6	Descartar un módulo ya construido que no cumplía los requisitos	**Falla interna**	No conformidad
7	Hotfix de urgencia por un error reportado por usuarios	**Falla externa**	No conformidad
8	Soporte atendiendo reclamos por un error en producción	**Falla externa**	No conformidad
9	Compensación económica por una caída del servicio	**Falla externa**	No conformidad
10	Auditoría de calidad interna del proyecto	**Evaluación**	Conformidad
11	Guías de codificación y diseño de arquitectura	**Prevención**	Conformidad
12	Pérdida de clientes por mala reputación	**Falla externa** (costo invisible, difícil de medir)	No conformidad

Detalle del 5: la corrección y la repetición de pruebas por haber corregido son falla interna; las pruebas planificadas del punto 3 son evaluación.

13.5. A.5 Ejercicios de complejidad ciclomática

13.5.1. Ejercicio 5: `clasificarPedido`

a) Decisiones: `if` (1), `else if` (2), `for` (3), `if` interno (4), `||` (5) y operador ternario (6). Complejidad = 6 + 1 = **7**. (Verificado con ESLint, que informa “complexity of 7”.)

b) Como mínimo **7 casos** de prueba para cubrir los caminos básicos.

c) Camino base (todas las decisiones en “falso”) y cambio de una decisión por vez:

N.º	Qué se activa	Entrada (`total`, `items`, `urgente`)	Resultado
1	Camino base	5000, vacío, falso	`"normal"`
2	`if total > 50000`	60000, vacío, falso	`"alto"`

N.º	Qué se activa	Entrada (`total`, `items`, `urgente`)	Resultado
3	`else if total > 10000`	20000, vacío, falso	`"medio"`
4	Entra al `for`; condición interna falsa (ambas partes del `||`)	5000, un ítem no agotado con cantidad 1, falso	`"normal"`
5	Primera parte del `||` verdadera	5000, un ítem agotado, falso	`"revisar"`
6	Segunda parte del `||` verdadera	5000, un ítem no agotado con cantidad 150, falso	`"revisar"`
7	Ternario verdadero	5000, vacío, verdadero	`"normal-urgente"`

Los resultados esperados fueron comprobados ejecutando la función.

13.5.2. Ejercicio 6: `costo_envio`

a) Decisiones: `if peso ⩽ 5` (1), `elif peso ⩽ 20` (2), `if urgente` (3), `if` de la zona (4) y el `and` (5).

- Contando el `and` como una decisión aparte: 5 + 1 = **6**.
- Contando la condición compuesta como **una sola** decisión: 4 + 1 = **5**.

b) Camino base: peso ≤ 5, no urgente, zona Centro. Después se cambia una decisión por vez:

N.º	Qué se activa	Entrada (`peso`, `urgente`, `zona`)	Resultado
1	Camino base	3, `False`, `"Centro"`	100
2	`elif peso ⩽ 20` verdadero	10, `False`, `"Centro"`	200
3	Rama `else` (peso > 20)	30, `False`, `"Centro"`	400
4	`urgente` verdadero	3, `True`, `"Centro"`	150
5	Zona “Patagonia”, pero `peso > 10` falso	3, `False`, `"Patagonia"`	100
6	Zona “Patagonia” y `peso > 10` verdadero	15, `False`, `"Patagonia"`	300

Si contás la condición compuesta como una sola decisión, bastan **5** casos (se puede omitir el caso 5). Los resultados fueron comprobados ejecutando la función.

> **Ojo.** Estos casos de prueba son *un* conjunto válido de caminos básicos; no es el único. Lo que importa es que sean **linealmente independientes** y que cada decisión quede ejercitada en verdadero y en falso.

13.6. A.6 Casos para analizar

13.6.1. Caso A: "Funciona, no lo toquemos"

a) Calidad interna deficiente (duplicación en 9 servicios, sin pruebas automáticas), **deuda técnica**, costos de la no calidad (retrabajo y errores en producción) y la idea de que *funcionar no significa estar bien construido.*

b) El **interés** es la diferencia entre lo estimado y lo real: 1 día contra 3 semanas, más dos errores en producción. Ese esfuerzo extra aparece *cada vez* que se toca esa zona.

c) Excesiva o arriesgada: nueve copias con diferencias y sin pruebas implican riesgo alto. Si fue voluntaria o involuntaria, **el caso no lo dice**, y conviene decirlo así. Lo preocupante es que la actitud del gerente ("no hay que tocar nada") muestra que **no existe un plan para saldarla**.

d) *Visibles:* las 3 semanas de trabajo (retrabajo) y la corrección de los dos errores en producción (incidentes y soporte). *Invisibles:* el miedo a tocar el código y el desgaste del equipo, el retraso de la campaña comercial (oportunidades perdidas) y la pérdida de confianza del área comercial.

e) Detectar (análisis estático: duplicación y complejidad); **registrar** (en el *backlog*, con ubicación, causa e impacto); **priorizar** (riesgo × impacto: primero lo que más se cambia y más dinero mueve); **reducir** (escribir pruebas y consolidar las 9 copias en un único cálculo, dentro de la capacidad del *sprint*). Y recordar que gestionar no es eliminar toda la deuda: es tomar decisiones conscientes y que no crezca sin control.

13.6.2. Caso B: "Quality Gate: Passed"

a) "Passed" significa que el **código nuevo** cumple las reglas de la puerta de calidad; **no dice nada del código existente**, que sigue con confiabilidad E (1.349 *bugs*), 78,8 % de duplicación y 46 % de cobertura. Evita que la deuda vieja empeore, pero no la paga.

b) Con el **semáforo verde** (cap. 6) y con la lectura de las capturas de SonarQube (cap. 5): **un indicador verde no cuenta toda la historia**. También con el principio 7: decidir con evidencia pertinente, confiable y *analizada en contexto.*

c) Mirar varias dimensiones: confiabilidad y vulnerabilidades, duplicación, cobertura, deuda técnica expresada en días, su **evolución en el tiempo** y, sobre todo, evidencia del producto en uso (incidentes y defectos escapados a producción).

13.6.3. Caso C: "Retrospectivas sin acciones"

a) *Principio 6, Mejora continua:* los mismos problemas reaparecen y las retrospectivas no generan acciones, así que el aprendizaje no se convierte en mejoras sostenidas. *Principio 7, Evidencia:* se cuentan *bugs*, pero nadie analiza las causas. *Principio 2, Liderazgo:* el CTO delega la calidad en QA y no asigna responsables ni recursos; "la calidad no se exige solamente: se lidera, se facilita y se demuestra". (La participación, principio 3, se cumple a medias: el equipo tiene voz, pero no la autonomía ni las condiciones para actuar.)

b) Confunde **QA con testing** y con un área. QA construye las condiciones para prevenir defectos; QC verifica el producto; y en *Quality Engineering* la calidad es **responsabilidad de todo el equipo**.

c) Un ciclo **PDCA** (o una retrospectiva con acuerdos y experimentos): *planificar* una acción concreta con responsable y fecha para "los viernes fallan los despliegues" (por ejemplo, pruebas automáticas del flujo crítico o no desplegar los viernes); *hacer* la prueba durante un sprint; *verificar* comparando incidentes antes y después; *actuar*, estandarizando si funcionó. Si el problema se repite, es medible y las causas no son evidentes, **DMAIC** permitiría hallar la causa raíz con datos (con un Ishikawa para ordenar las hipótesis).

14. Anexo B. Glosario

Término	Significado breve	Cap.
Calidad (ISO 9000:2015)	Grado en que un conjunto de características inherentes de un objeto cumple los requisitos	1
Característica inherente	Propiedad que el objeto tiene en sí (rendimiento, usabilidad, seguridad)	1
Requisito	Necesidad, expectativa u obligación	1
Calidad de software (IEEE)	Grado en que un sistema, componente o proceso cumple los requerimientos especificados y las necesidades o expectativas del usuario	2
Perspectiva de producto	Calidad en las características medibles del software	2
Perspectiva de proceso	Calidad en desarrollar con procesos y especificaciones definidos	2
Perspectiva de personas usuarias	Calidad = satisface a quien lo usa en su contexto real	2
Perspectiva basada en valor	Equilibrio entre beneficio, costo y restricciones	2
Perspectiva trascendental	Calidad como excelencia, difícil de definir pero que se reconoce	2
Calidad del producto	Grado en que el producto cumple requisitos y satisface a las personas usuarias	3
Calidad del proceso	Grado en que las prácticas de desarrollo están definidas, se aplican, se controlan y mejoran	3
QA (Quality Assurance)	Prevenir defectos desde el proceso y las prácticas	3
QC (Quality Control)	Verificar el producto y detectar defectos	3
Quality Engineering	Enfoque donde la calidad es responsabilidad de todo el equipo	3
Definition of Done (DoD)	Criterios acordados que debe cumplir un trabajo para considerarse terminado	3
Quality gate	“Puerta de calidad”: reglas que el código debe cumplir para avanzar	3, 5
Calidad externa / interna	Lo que ve el usuario / cómo está construido el código	4
Mantenibilidad	Facilidad con que el software se entiende, corrige y modifica	4

Término	Significado breve	Cap.
Complejidad ciclomática	Estimación de los caminos linealmente independientes; decisiones + 1	4
Camino básico	Camino independiente que aporta una decisión nueva; base del testing de caja blanca	4
Refactorizar	Mejorar la estructura interna del código sin cambiar lo que hace	4, 5
Code smell	Síntoma en el código que sugiere un mal diseño	5
Análisis estático	Revisar el código con una herramienta, sin ejecutarlo	4, 5
Deuda técnica	Deficiencias acumuladas en calidad interna que encarecen modificar y mantener	5
Interés (de la deuda)	Esfuerzo adicional que exige cada cambio por la deuda	5
Costo visible / invisible	Los que aparecen en horas, facturas o tickets / los difíciles de medir (confianza, reputación, desgaste)	5
Semáforo verde	Indicadores de gestión en verde que ocultan un producto deficiente	6
SGC (Sistema de Gestión de la Calidad)	Forma sistemática de dirigir y controlar la organización para asegurar y mejorar la calidad	6, 8
Certificación	Reconocimiento de que un SGC cumple una norma; respalda al proceso, no a cada producto	6
Costos de la calidad	Prevención + evaluación + fallas internas + fallas externas	7
Prevención	Costos para evitar que los defectos aparezcan	7
Evaluación	Costos de comprobar que el producto cumple (pruebas, auditorías)	7
Falla interna	Defecto detectado antes de entregar	7
Falla externa	Defecto detectado por los usuarios	7
Costos de conformidad	Prevención + evaluación (inversión planificada)	7
Costos de no conformidad / COPQ	Fallas internas + externas (*Cost of Poor Quality*)	7
Escape rate	Porcentaje de defectos que llegó a producción	7
DRE	Eficiencia de remoción de defectos: porcentaje eliminado antes de producción	7
Tasa de retrabajo	Horas de retrabajo sobre horas totales	7
Calidad Total	Excelencia que satisface al cliente interno y externo, al menor costo, en armonía con el entorno social, en un proceso continuo	8

Término	Significado breve	Cap.
Cliente interno / externo	Quien recibe mi trabajo dentro / fuera de la organización	8
PDCA	Planificar, Hacer, Verificar, Actuar (ciclo de Deming)	8
Kaizen	Mejora continua mediante pequeños cambios con participación de todos	8
Six Sigma / DMAIC	Metodología estadística: Definir, Medir, Analizar, Mejorar, Controlar	8
Retrospectiva	Reunión periódica del equipo para revisar y mejorar su forma de trabajar	8
Ishikawa	Diagrama que organiza causas posibles de un problema por categorías	8
Pareto	Pocas causas explican la mayoría de los efectos (80/20)	8
5 porqués	Preguntar "¿por qué?" repetidamente hasta la causa raíz	8
Triangulación	Contrastar un dato con otras fuentes o métodos	8
Prueba A/B	Comparar dos variantes con usuarios reales	8
SLA	Acuerdo de nivel de servicio con un proveedor	8
CSAT / CES	Satisfacción del cliente / esfuerzo que le costó lograr su objetivo	8

15. Anexo C. Resumen y fórmulas

15.1. C.1 Resumen

Tema	Lo esencial
Calidad	Grado en que las características inherentes de un objeto cumplen los requisitos (ISO). Relativa a quién y a qué requisitos. Crosby: lo caro es la no calidad.
Calidad de software	IEEE: requerimientos especificados **y** necesidades o expectativas. Lo pedido $\neq$ lo necesario.
5 perspectivas	Producto (medible) · Proceso (cómo se desarrolla) · Personas usuarias (le sirve) · Valor (equilibrio beneficio/costo) · Trascendental (excelencia).
Producto y proceso	El proceso crea las condiciones; el producto confirma la calidad. Influye, no garantiza.
QA / QC	QA previene (proceso); QC verifica (producto). QA $\neq$ testing.
Calidad interna	Funciona $\neq$ está bien construido. Se nota al cambiar.
Complejidad ciclomática	Decisiones + 1 = caminos independientes = mínimo de casos. Meta: unidades simples, testeables, comprensibles.
Deuda técnica	Deficiencias de calidad interna que encarecen el cambio; interés = esfuerzo extra. Cuadrante; iceberg; Detectar, Registrar, Priorizar, Reducir.
Proceso $\neq$ calidad	Semáforo verde y empresa certificada: la certificación respalda al proceso; el producto se verifica en el producto.
Costos	Prevención + evaluación = conformidad; fallas internas + externas = no conformidad (COPQ). Si dominan las fallas externas: prevenir y detectar antes.
Calidad Total	Excelencia, cliente interno y externo, menor costo, entorno social, mejora continua; objetivo: supervivencia.
8 principios	1 Cliente · 2 Liderazgo · 3 Participación · 4 Procesos · 5 Enfoque de sistema / SGC · 6 Mejora continua · 7 Evidencia · 8 Proveedores.

15.2. C.2 Fórmulas

Qué	Fórmula
Complejidad ciclomática	decisiones + 1 (ó $E - N + 2P$ en el grafo)

Qué	Fórmula
COPQ (no conformidad)	Fallas internas + Fallas externas
Conformidad / Costo total de la calidad	Prevención + Evaluación / Conformidad + No conformidad
COPQ sobre ingresos	COPQ / Ingresos x 100
Origen del COPQ	(Fallas internas o externas) / COPQ x 100
Costo por cliente	COPQ / Clientes activos
COPQ sobre el proyecto	(Fallas internas + externas) / Costo total del proyecto x 100
Costo promedio por defecto	Costo total asociado a fallas / Cantidad de defectos
Escape rate	Defectos en producción / Total de defectos detectados x 100
DRE	Defectos eliminados antes de producción / Total de defectos detectados x 100
Tasa de retrabajo	Horas de retrabajo / Horas totales del proyecto x 100
Relación entre indicadores	DRE + *Escape rate* = 100 % (aporte mío)

> **Ojo.** Antes de calcular un porcentaje, preguntate **"¿sobre qué?"**: ingresos, presupuesto, costo total de la calidad y COPQ dan resultados distintos.

15.3. C.3 Frases para llevarse

- "La calidad **no cuesta**: lo que cuesta es **no hacer las cosas bien la primera vez**." (Crosby)
- "**El proceso crea las condiciones. El producto confirma la calidad.**"
- "**Cumplir el proceso no garantiza la calidad.**"
- "**Que funcione no significa que esté bien construido.**"
- "**Lo que no se corrige hoy se paga mañana… con intereses.**"
- "Un indicador aislado describe una señal. **Su evolución** permite gestionar la calidad."
- "**La mayor oportunidad no está en evaluar más, sino en prevenir y detectar antes.**"
- "La calidad no se exige solamente: se **lidera**, se **facilita** y se **demuestra**."

16. Anexo D. Qué preguntó el parcial 2025 de esta unidad

Para armar este anexo cruzé el modelo de examen **"Examen Parcial 1 - 2025"** (que tenés cargado en el proyecto) con los capítulos del apunte. **No tengo la clave de corrección oficial**: las respuestas son mi análisis a partir de las slides.

16.1. D.1 Preguntas de teoría que son de la Unidad I

N.º	Tema	Respuesta	Dónde se estudia
1	Perspectiva "basada en el producto"	**a**: características inherentes y **medibles** (como la complejidad ciclomática o las líneas de código)	2.3
2	Qué define correctamente la deuda técnica	**d** (trabajo adicional acumulado por elegir la solución fácil a corto plazo). Ver nota abajo	5.1
3	Enfoque al cliente en TQM	**c**: entender y satisfacer necesidades **actuales y futuras** de clientes **internos y externos**	8.4
4	Cuáles son costos de la calidad	**a, b y c** (formación = prevención; inspecciones y revisiones = evaluación; pérdida de ventas por reputación = falla externa)	7.2 y 7.3
7	Complejidad ciclomática en caja blanca	La opción que dice **"determinar el número mínimo de caminos independientes a probar"**	4.3
8	Pilar de la Calidad Total según Deming	**a**: mejora continua basada en el ciclo **PDCA**	8.9

Notas.

- **Pregunta 2.** La definición de la cátedra (deficiencias de calidad interna que encarecen el cambio, con "interés") coincide con la opción **d**. La **a** (licencias) es claramente falsa. La **c** (falta de documentación) puede ser una señal, pero no define la deuda. La **b** ("código difícil de mantener, que necesita actualizar sus librerías o que está roto") describe síntomas y parte de ella se parece a lo que dice la cátedra ("más costoso mantenerlo"), pero "está roto" apunta más a un *bug* que a deuda. Como la consigna dice "qué **afirmaciones**" (plural), es posible que la clave acepte más de una. Si podés, **consultalo con la cátedra**; si no, la opción segura es la **d**.

- **Pregunta 7.** En el documento las opciones aparecen rotuladas con las letras *e, f, g, h* (error de numeración del examen). La correcta es la **primera**: determinar el número mínimo de caminos independientes a probar.
- **Pregunta 4.** Como dice "marcar la/s respuesta/s correcta/s", era una pregunta con **más de una opción correcta**.

16.2. D.2 Preguntas del mismo examen que no son de esta unidad

N.º	Tema	Unidad
5	Principio fundamental del testing	Fundamentos de las pruebas (Unidad II)
6	Qué NO es técnica de caja negra	Diseño de casos de prueba
9	Orden de los niveles de testing	Fundamentos de las pruebas
10	Verificación vs. validación	Fundamentos de las pruebas (se conecta con "lo pedido vs. lo necesario", ver 2.2)
Práctica 1	EcoMeter: clases de equivalencia	Diseño de casos de prueba
Práctica 2	`costo_envio`: caminos básicos	Caja blanca (usa la complejidad ciclomática del cap. 4)

Además, el modelo de examen **2022** que está en el proyecto (*Calidad del Producto y del Proceso de SW*) tiene teoría de testing, niveles, tipos de pruebas, defectos y caja blanca: **ninguna pregunta es de esta unidad**.

16.3. D.3 Qué conclusión sacar

- En el modelo 2025, **5 de las 10 preguntas teóricas** son de la Unidad I (más la complejidad ciclomática, que toca otra unidad). Es un buen indicio del peso de este material, pero **no garantiza** lo que tomen en 2026.
- El estilo es de **opción múltiple con definiciones y formulaciones de la cátedra** (a veces con varias correctas). Por eso conviene memorizar las frases ancla del Anexo C y la tabla de reflejos del capítulo 9.
- Prioridad de estudio que te sugiero (opinión mía, según lo que se preguntó y lo que desarrollan las slides): **(1)** las cinco perspectivas, producto vs. proceso y QA vs. QC; **(2)** complejidad ciclomática (saber calcularla); **(3)** deuda técnica; **(4)** costos de la calidad (clasificar y calcular); **(5)** los ocho principios y el PDCA, con sus casos.
- Los casos con preguntas (semáforo verde, empresa certificada, ejemplos de los principios) son típicos de la modalidad **teórico-práctica**: practicalos con la plantilla del apartado 11.3.